

HPC 优化原理篇：CPU 与 GPU 的理论基础

目 录

1 Part I: CPU 侧原理	5
1.1 MoE 架构与 W8A8 量化原理	5
1.2 引言：为什么大模型都在用 MoE	5
1.3 从稠密 FFN 到稀疏专家	5
1.3.1 FFN: Transformer 里最“重”的部件	5
1.3.2 MoE 的核心想法	5
1.3.3 参数量与计算量的对比	6
1.4 DeepSeek-V3 的两个标志性设计	6
1.4.1 更细粒度的专家	6
1.4.2 共享专家	6
1.5 单层 MoE 前向流程	7
1.5.1 阶段一：路由打分	7
1.5.2 阶段二：Top-K 选择（带偏置）	7
1.5.3 阶段三：路由权重归一化	8
1.5.4 阶段四：激活量化	8
1.5.5 阶段五：SwiGLU 专家计算	8
1.5.6 阶段六：加权合并	9
1.6 W8A8 量化推理详解	10
1.6.1 为什么用 INT8	10
1.6.2 对称量化与 scale 策略	10
1.6.3 $\text{INT8} \times \text{INT8} \rightarrow \text{INT32} \rightarrow \text{FP32}$ 数据通路	10
1.6.4 手算：一个三维 W8A8 点积	10
1.7 隐藏层重量化与误差传播	11
1.8 思考题 1：算术强度分析	11
1.8.1 乘加次数估算	11
1.8.2 访存量估算（按 token 遍历）	12
1.8.3 算术强度	12
1.8.4 按专家分组后	12
1.9 思考题 3：INT8 累加溢出分析	13
1.9.1 单个乘积的最大值	13
1.9.2 INT16 何时溢出	13
1.9.3 INT32 的余量	13
1.10 SIMD/VNNI/AMX 指令原理	14
1.11 引言：从自动向量化到手写 SIMD	14

1.12 AVX 寄存器与数据类型	14
1.12.1 寄存器层级	14
1.12.2 C++ 数据类型	14
1.12.3 intrinsic 命名规则	15
1.13 VNNI 指令详解	15
1.13.1 vpdpwssd: $\text{int16} \times \text{int16} \rightarrow \text{int32}$	15
1.13.2 vpdpbussd: $\text{uint8} \times \text{int8} \rightarrow \text{int32}$	16
1.14 signed $\times$ signed int8 的两种方案	16
1.14.1 方案 A: int8 扩展到 int16 + vpdpwssd	16
1.14.2 方案 B: 偏移技巧 + vpdpbussd	16
1.15 AMX 编程模型与 Tile 寄存器	17
1.16 引言: 从 VNNI 到 AMX	17
1.17 AMX 编程模型	17
1.17.1 Tile 寄存器	17
1.17.2 Tile 配置 (Palette)	18
1.18 核心 AMX 指令	18
1.18.1 加载: <code>_tile_loadd</code>	18
1.18.2 矩阵乘加: <code>_tile_dpbssd</code>	19
1.18.3 存储: <code>_tile_stored</code>	19
1.18.4 清零: <code>_tile_zero</code>	19
1.19 Top-down 与 Roofline 模型	19
1.20 引言: 从“哪里慢”到“为什么慢”	19
1.21 CPU 流水线模型	19
1.22 Top-down 四分类	20
1.23 Back-End Bound 细分	20
1.23.1 Lab2 中的典型 Top-down 结果	21
1.23.2 优化后的预期变化	21
1.24 Roofline 模型	21
1.24.1 基本概念	21
1.24.2 Lab2 的 Roofline 分析	22
1.24.3 估算运算量与数据搬运量	22
1.25 IPC 与其他指标	22
1.26 性能上限分析	23
2 Part II: GPU 侧原理	23
2.1 Linear Attention 与 Gated DeltaNet	23
2.2 引言: 为什么需要 Linear Attention	23
2.3 标准注意力的代价分析	23
2.4 Linear Attention: 用状态递推替代 $L \times L$ 矩阵	24
2.4.1 从 softmax 到特征映射	24
2.4.2 状态递推形式	24
2.4.3 复杂度与局限	25
2.5 GLA: 加入遗忘门	25

2.6 DeltaNet: delta rule 定向覆盖	25
2.7 Gated DeltaNet: 门控遗忘与 delta update 的组合	26
2.8 计算接口与数据约定	26
2.9 Chunk-wise Parallel 推导	27
2.10 引言: 在递推与并行之间折中	27
2.11 递推的并行困境: 为什么逐 token 难以并行	27
2.12 完全并行形式的代价: $L \times L$ 矩阵的诅咒	28
2.13 Chunk-wise 的核心思想: 切一刀在中间	29
2.14 从 Linear Attention 到 chunk-wise 公式	29
2.14.1 Linear Attention 的递推回顾	29
2.14.2 chunk 内输出的拆分	29
2.15 GDN 的完整 chunk-wise 公式	30
2.16 逐步推导直觉: A、U、W 从何而来	30
2.16.1 A 矩阵的来源: 因果依赖的逆	30
2.16.2 U 与 W 的物理意义: 预处理中间量	31
2.16.3 状态递推如何结合 chunk 内信息	31
2.17 chunk size C 的选择权衡	32
2.18 手算例子: 用小数值走一遍 chunk-wise	32
2.19 GPU 存储层次与 Bank Conflict	34
2.20 引言: 为什么存储层次决定 GDN 性能	34
2.21 GPU 存储层次总览	34
2.22 Global Memory: 高延迟的“远郊仓库”	34
2.23 L2 Cache: 硬件管理的“二级缓存”	35
2.24 Shared Memory: 程序员显式管理的高速缓存	35
2.24.1 大小配置	35
2.24.2 在 TileLang 中分配	35
2.25 Bank Conflict: shared memory 的隐形杀手	36
2.25.1 32-bit stride 访问的常见陷阱	36
2.25.2 解决方法: padding 与交错访问	36
2.26 Register: 每线程私有的最快存储	36
2.27 在 GDN 中的应用: 各张量该放哪里	37
2.28 Kernel Fusion 与 Ping-pong 原理	38
2.29 引言: 朴素实现的困境	38
2.30 Kernel Launch 开销	38
2.31 中间结果的访存代价	38
2.32 Kernel Fusion 原理	39
2.32.1 可以融合的部分	39
2.32.2 不能融合的部分	39
2.33 等价数学变换	40
2.34 ping-pong buffer: 双缓冲原理	40
2.35 同步与正确性	40
2.36 multi-buffering: 更深的流水线	41

2.37 手算例子：时间线对比	41
2.37.1 加速比分析	42
2.38 Warp Specialization 与 Tensor Core	42
2.39 引言：GDN 的两类计算	42
2.40 GPU 的两类计算单元	42
2.40.1 CUDA Core：标量逐元素计算	42
2.40.2 Tensor Core：矩阵乘加	42
2.40.3 两类单元的吞吐对比	43
2.41 Tensor Core 编程	43
2.42 Warp：GPU 的执行单元	43
2.43 Warp Specialization 思想	44
2.43.1 Producer 与 Consumer	44
2.43.2 warp 数量的分配	44
2.44 在 GDN 中的应用	44
2.44.1 为什么要分两个 consumer warp	45
2.45 同步机制	45
2.46 手算例子：重叠收益	45
2.46.1 多 chunk 的流水线效果	46
2.47 寄存器压力与 occupancy	46
2.47.1 如何判断 occupancy 是否足够	46

1 Part I: CPU 侧原理

1.1 MoE 架构与 W8A8 量化原理

1.2 引言：为什么大模型都在用 MoE

从 GPT-4 到 DeepSeek-V3、Qwen3、GLM-4，几乎所有前沿大模型都采用了 **MoE** (Mixture of Experts, 混合专家) 架构。一个自然的问题是：模型参数越多越聪明，但参数多了计算就慢，这个矛盾怎么破？MoE 给出的答案是，让总参数量很大，但每个 token 只激活其中一小部分来参与计算。

本教程是 Lab2 系列的第一篇，目标是把 MoE 前向计算的每一步在数学和工程层面都讲透。你读完后应该能：手算一个量化的点积、推算一次前向的访存量与计算量、回答 Lab2 的三道思考题中的两道（第 1 题和第 3 题），并为后续的向量化优化建立清晰的算法心智模型。

直觉 不妨把 MoE 想象成一家大医院。医院里有眼科、骨科、心内科等几十个科室（专家），但每个病人（token）来了之后，分诊台（路由器）只把他送到最合适的两三个科室去诊治，而不是让所有科室都给他看一遍。医院的“总能力”是所有科室之和，但每个病人消耗的资源远小于此。

1.3 从稠密 FFN 到稀疏专家

1.3.1 FFN: Transformer 里最“重”的部件

Transformer 的每一层由 **Attention**（自注意力）和 **FFN**（前馈网络）两部分组成。Attention 负责让序列中不同位置的信息相互混合，FFN 则负责对每个位置的表示做非线性变换。在参数量上，FFN 通常占据整层参数的三分之二以上。

本实验中的专家采用 **SwiGLU** 结构，它包含三个线性投影：

$$\text{FFN}(x) = W_d(\text{SiLU}(W_g x) \odot W_u x) \quad (1.1)$$

其中 W_g, W_u 把 D 维输入投影到 H 维隐藏空间， W_d 再投回 D 维； $\text{SiLU}(v) = \frac{v}{1+e^{-v}}$ 是激活函数， $\odot$ 是逐元素相乘。忽略偏置时，一个 SwiGLU 专家的权重数为

$$P_{\text{expert}} = HD + HD + DH = 3DH. \quad (1.2)$$

1.3.2 MoE 的核心想法

MoE 的出发点是：与其让所有 token 共享一个大 FFN，不如准备 E 个“专家”FFN，让每个 token 只经过其中得分最高的 K 个（ $K \Rightarrow E$ ）。这样，总参数量可以显著增长，而每个 token 只激活少量专家，从而在一定程度上分离模型容量与激活层的大小，也就是单 token 计算量和其占用的内存。

但天下没有免费的午餐：路由和分发会带来额外的控制开销，而且所有专家的权重都要驻留在内存中（推理时），这对内存带宽提出了挑战。这正是 Lab2 要优化的痛点。

本实验采用 DeepSeek-V3 的 MoE 架构。它与早期 MoE（如 Mixtral 的 8 专家选 2）相比有两个标志性设计：更细粒度的专家（256 选 8），以及一个所有 token 都必须经过的共享专家。下面逐一展开。

1.3.3 参数量与计算量的对比

我们用具体数值来感受 MoE 的优势。取 DeepSeek-V3 风格的设置 $D = 256$ 、 $H = 128$ ，共享专家加上 16 个路由专家（共 17 个专家），每个 token 选 $K = 4$ 个路由专家加 1 个共享专家。

代入 $3 \times 256 \times 128 = 98\,304$ ，每个专家有 98 304 个权重。17 个专家一共包含 1 671 168 个权重，但每个 token 只激活 4 个路由专家和 1 个共享专家，对应的矩阵乘加数为

$$(K + 1) \times 3DH = 5 \times 98\,304 = 491\,520 \text{ MACs.} \quad (1.3)$$

若稠密 FFN 具有相同的总参数量，其中间维度需要取为 $H_{\text{dense}} = 17H = 2176$ ，此时每个 token 都必须经过全部参数，对应的乘加数为 $3DH_{\text{dense}} = 1\,671\,168$ MACs。相比之下，MoE 每个 token 只计算 5 个激活专家，计算量仅为稠密 FFN 的

$$\frac{(K + 1) \times 3DH}{17 \times 3DH} = \frac{5}{17} \approx 29.4\%. \quad (1.4)$$

项目	公式	数值
单专家参数量	$3DH$	98 304
总专家参数量	$17 \times 3DH$	1 671 168
激活参数量	$(K + 1) \times 3DH$	491 520
稠密 FFN 等效中间维	$H_{\text{dense}} = 17H$	2176
稠密 FFN MACs	$3DH_{\text{dense}}$	1 671 168
MoE / 稠密计算比	$\frac{K+1}{17}$	$\approx 29.4\%$

Table 1: MoE 与等效稠密 FFN 的参数量与计算量对比

也就是说，在总专家参数量相同的情况下，MoE 将每个 token 的专家计算量降低了约 70.6%，理论上约为稠密 FFN 的 $\frac{1}{3.4}$ 。这体现了 MoE 的核心优势：保留较大的总参数容量，同时只激活其中一小部分参与当前 token 的计算。

1.4 DeepSeek-V3 的两个标志性设计

1.4.1 更细粒度的专家

早期 MoE（如 Mixtral）通常用 8 个较大的专家选 2 个。DeepSeek-V3 走了另一条路：用更多更小的专家（256 个路由专家选 8 个）。直觉上，专家越小、数量越多，同样的总参数量下专家组合的可能性大大增加，模型能更精细地为不同 token 匹配合适的专家组合。

直觉 如果把一个大专家拆成两个小专家，原来所有 token 都被迫经过这个大专家的同一套变换；拆分后，有的 token 走左半边、有的走右半边，模型的表达能力更丰富了。拆得越细，组合空间越大，但这要求路由器更聪明。

1.4.2 共享专家

DeepSeek-V3 还设计了一个所有 token 都必须经过的**共享专家**（shared expert）。它的职责是学习通用知识，让路由专家专注于特化知识。从工程角度看，共享专家对每个 token 都执行，不经过路由选择，所以它的访存模式是确定性的，优化起来相对简单。

本实验中，每个 token 的前向流程是：先经过共享专家，再经过 K 个被路由选中的路由专家，最后加权合并并加上残差。

1.5 单层 MoE 前向流程

现在我们把完整的单层 MoE 前向拆成六个阶段，对照参考实现 `moe_ref.cpp` 中的代码逐步讲解。先给出整体数据流图，再逐段展开。

token x_t (FP32)
 → 路由打分: $s_e = \sigma(r_e^T x_t)$ (FP32)
 → Top-K 选择: 按 $s_e + b_e$ 选 K 个专家
 → 路由权重归一化: $g_e = s_e / \sum_{j \in \mathcal{S}_t} s_j$
 → 激活量化: $x_q = \text{round}(x_t / s_x)$ (FP32 $\rightarrow$ INT8)
 → 共享专家 + 被选中的 K 个路由专家 (INT8 权重 $\times$ INT8 激活 $\rightarrow$ INT32 $\rightarrow$ FP32)
 → 加权合并 + 残差: $y_t = x_t + o_{\text{shared}} + \sum g_e o_e$

Figure 1: 单 token 的 MoE 前向数据流

1.5.1 阶段一：路由打分

对每个 token $x_t \in \mathbb{R}^D$ ，先计算它与每个路由专家 e 的路由 logit，再经 **sigmoid** 得到亲和度 (affinity)：

$$z_{t,e} = r_e^T x_t, \quad s_{t,e} = \sigma(z_{t,e}) = \frac{1}{1 + e^{-z_{t,e}}}, \quad e = 1, \dots, E. \quad (1.5)$$

其中 r_e 是路由矩阵 w_{router} 的第 e 行。sigmoid 把内积映射到 $(0,1)$ 区间，值越大表示该 token 与专家 e 越“匹配”。这一步全在 FP32 中进行。

对应代码 (`moe_ref.cpp`):

```
float s[MAX_NUM_EXPERTS];
for (int e = 0; e < num_experts; e++) {
    float acc = 0.0f;
    for (int d = 0; d < d_model; d++) {
        acc += w.w_router[(size_t)e * d_model + d] * xt[d];
    }
    s[e] = 1.0f / (1.0f + expf(-acc));
}
```

逐元素来看：第 e 个专家的内积 $\text{acc} = r_e \cdot x_t$ ，再经 sigmoid 得到亲和度 $s[e]$ 。

1.5.2 阶段二：Top-K 选择（带偏置）

定义被选中的专家集合

$$\mathcal{S}_t = \text{TopK}_{e \in \{1, \dots, E\}}(s_{t,e} + b_e), \quad \text{abs}(\mathcal{S}_t) = K. \quad (1.6)$$

这里有一个关键细节：偏置 b_e 只用于**选择**，不进入后续的归一化权重。这是 DeepSeek-V3 的**辅助损失无关的负载均衡** (auxiliary-loss-free load balancing) 设计：通过调整 b_e 让 token 均匀地流向各专家，但一旦选中，gate 值仍由原始 sigmoid 亲和度决定，避免偏置扭曲了专家的输出权重。

实现中用了一个朴素的选择：对每个 k 从 0 到 $K-1$ ，遍历所有未选中的专家找出当前最大的。时间复杂度 $O(KE)$ ，因为 K 和 E 都不大，这里不是瓶颈，不需要优化。

1.5.3 阶段三：路由权重归一化

只对 $\mathcal{S}_t$ 中的原始亲和度（不含偏置）做归一化：

$$g_{t,e} = \begin{cases} \frac{s_{t,e}}{\sum_{j \in \mathcal{S}_t} s_{t,j}} & \text{if } e \in \mathcal{S}_t \\ 0 & \text{if } e \notin \mathcal{S}_t. \end{cases} \quad (1.7)$$

直觉 为什么要归一化？因为不同 token 选中的专家组合不同，sigmoid 值的绝对大小没有可比性。归一化后，被选中的 K 个专家的 gate 之和恒为 1，保证每个 token 的输出尺度一致。注意偏置 b_e 在这里完全不参与，它只影响“谁被选中”，不影响“选中后权重多大”。

1.5.4 阶段四：激活量化

真实推理引擎很少用 FP32 存储和计算专家权重：INT8 量化能把内存占用和带宽需求降到 1/4，同时现代 SIMD 指令集处理 INT8 的吞吐量远高于 FP32。本实验采用 **W8A8**（权重和激活都量化到 INT8）策略，对激活使用 **per-token scale**，每个 token 向量单独计算缩放因子：

$$x_q = \text{round}\left(\frac{x}{s_x}\right), \quad s_x = \frac{\max_i \text{abs}(x_i)}{127}. \quad (1.8)$$

这是**对称量化**：零点固定为 0，只需一个 scale。量化后 x_q 是 INT8（-128 到 127），反量化时 $x \approx x_q \times s_x$ 。

对应代码：

```
float x_amax = 0.0f;
for (int d = 0; d < d_model; d++) {
    float a = fabsf(xt[d]);
    if (a > x_amax) x_amax = a;
}
float s_x = (x_amax > 0.0f) ? x_amax / 127.0f : 1.0f;
int8_t xq[MAX_D_MODEL];
for (int d = 0; d < d_model; d++) {
    xq[d] = (int8_t)lrintf(xt[d] / s_x);
}
```

先求 $\max(\text{abs}(x))$ 得到 s_x ，再把每个元素 round 到 INT8。

1.5.5 阶段五：SwiGLU 专家计算

共享专家和每个被选中的路由专家都是 SwiGLU 结构，计算流程分三步。关键点是：INT8 权重乘 INT8 激活在 INT32 中累加，再乘以 scale 反量化回 FP32 做非线性运算；中间结果还要**重新量化**（requantize）回 INT8，才能进入下一层 INT8 矩阵乘。


```

// gate / up 投影 + SwiGLU
float h[MAX_D_FF];
float h_amax = 0.0f;
for (int f = 0; f < d_ff; f++) {
    int32_t acc_g = 0, acc_u = 0;
    for (int d = 0; d < d_model; d++) {
        acc_g += (int32_t)w_gate[f * d_model + d] * (int32_t)xq[d];
        acc_u += (int32_t)w_up[f * d_model + d] * (int32_t)xq[d];
    }
    float vg = (float)acc_g * (s_x * s_gate);
    float vu = (float)acc_u * (s_x * s_up);
    float silu = vg / (1.0f + expf(-vg));
    h[f] = silu * vu;
    float a = fabsf(h[f]);
    if (a > h_amax) h_amax = a;
}
// 重新量化隐藏激活到 INT8
float s_h = (h_amax > 0.0f) ? h_amax / 127.0f : 1.0f;
int8_t hq[MAX_D_FF];
for (int f = 0; f < d_ff; f++) {
    hq[f] = (int8_t)lrintf(h[f] / s_h);
}
// down 投影
for (int d = 0; d < d_model; d++) {
    int32_t acc = 0;
    for (int f = 0; f < d_ff; f++) {
        acc += (int32_t)w_down[d * d_ff + f] * (int32_t)hq[f];
    }
    out[d] = (float)acc * (s_h * s_down);
}

```

逐段批注：gate/up 投影做 $\text{INT8} \times \text{INT8} \rightarrow \text{INT32}$ 累加，再乘 $(s_x \cdot s_{\text{gate}})$ 反量化回 FP32，经 SwiGLU 逐元素运算。requantize 阶段求 $\max(\text{abs}(h))$ ，round 回 INT8（注意 h_q 可能为负）。down 投影再次做 $\text{INT8} \times \text{INT8} \rightarrow \text{INT32}$ 累加，最后乘 $(s_h \cdot s_{\text{down}})$ 反量化得到输出。

直觉 为什么要中间 requantize？因为 gate/up 投影的输出经过 SwiGLU 后，数值范围会变化，原来的 s_x 已经不适用了。重新量化让 down 投影能继续用 INT8 矩阵乘，保持整条流水线的高吞吐。但这也带来一个副作用：浮点非线性运算（SiLU、逐元素乘）在重量化 round 边界上的微小差异，会让一个 INT8 值差 ± 1 ，再经 down 投影摊到整行输出上。这就是 check_result 用相对 L2 误差而非逐元素比对的原因。

1.5.6 阶段六：加权合并

最后，把所有专家的输出加权合并并加上残差：

$$y_t = x_t + \text{FFN}_{\text{shared}}(x_t) + \sum_{e \in \mathcal{S}_t} g_{t,e} \text{FFN}_e(x_t). \quad (1.9)$$

共享专家的 gate 始终为 1（它对所有 token 都执行，不参与归一化）。残差连接 x_t 让梯度能直接回流，是 Transformer 的标配。

1.6 W8A8 量化推理详解

1.6.1 为什么用 INT8

量化的动机有三：第一，INT8 权重占 1/4 的内存和带宽（FP32 占 4 字节，INT8 占 1 字节），这对访存密集型的 MoE 收益巨大；第二，现代 SIMD 指令集（如 AVX-512 VNNI、Intel AMX）处理 INT8 的吞吐量远高于 FP32，一条指令能做更多乘法；第三，W8A8 是精度与速度的折中，W4A16 等更激进方案精度损失更大且硬件支持有限。

1.6.2 对称量化与 scale 策略

本实验采用**对称量化**：零点固定为 0，只用一个正实数 scale 描述“INT8 的单位 1 对应多少 FP32”。权重每个矩阵一个 scale（per-matrix），激活每个 token 一个 scale（per-token）。反量化时把累加结果乘回两个 scale：

$$Wx \approx (W_q x_q) \cdot s_W s_x. \quad (1.10)$$

思考题第 2 题问：为什么激活用 *per-token scale*，而权重每个矩阵一个 *scale* 就够了？直觉上，同一批 token 的数值范围可能差异巨大（一个 token 在某维很大，另一个很小），共享一个 *scale* 会让小数值 token 的量化分辨率严重下降。而一个权重矩阵在整个推理过程中数值分布相对稳定，一个 *scale* 足以覆盖其动态范围。

1.6.3 INT8×INT8 → INT32 → FP32 数据通路

这是整个量化推理的核心。单次点积 $w^T x$ 的计算流程是：

1. INT8 乘法： $w_{q[i]} \times x_{q[i]}$ ，结果在 $[-128 \times 127, 127 \times 127] = [-16\,256, 16\,129]$ 范围内，需要 INT16 存放
2. INT32 累加：把所有 D 个乘积加起来，得到 $a_{\text{int}} = \sum_i w_{q[i]} x_{q[i]}$
3. 反量化： $y \approx a_{\text{int}} \times s_W \times s_x$ ，回到 FP32

直觉 为什么要 INT32 累加而不是 INT16？因为 D 可能高达 1024，1024 个 $127 \times 127 \approx 16\,129$ 相加可达 1.6×10^7 ，远超 INT16 的上限 32 767。INT32 上限约 2.1×10^9 ，留了两个数量级的余量。具体的溢出分析见后文。

1.6.4 手算：一个三维 W8A8 点积

我们用一个 $D = 3$ 的微型例子把整条流程走一遍，让你看清每一步的数值变化。

例 输入向量 $x = [1.00, -0.49, 0.25]^T$ ，权重矩阵中的一行 $w = [0.40, -0.80, 1.20]^T$ 。

第一步：量化激活。 $\max(\text{abs}(x)) = 1.00$ ，所以 $s_x = 1.00/127$ 。

$x_q = \text{round}([1.00, -0.49, 0.25]^T / (1/127)) = \text{round}([127, -62.23, 31.75]^T) = [127, -62, 32]^T$

第二步：量化权重。 假设 w 所在矩阵的最大绝对值为 1.20，则 $s_W = 1.20/127$ 。

$$w_q = \text{round}([0.40, -0.80, 1.20]^T / (1.20/127)) = \text{round}([42.33, -84.67, 127]^T) = [42, -85, 127]^T.$$

第三步：INT32 点积。

$$a_{\text{int}} = w_q^T x_q = 42 \times 127 + (-85) \times (-62) + 127 \times 32 = 5334 + 5270 + 4064 = 14\,668$$

第四步：反量化。

$$\hat{y} = a_{\text{int}} \times s_W \times s_x = 14\,668 \times (1.20/127) \times (1/127) \approx 1.09130. \quad (1.14)$$

对比 FP32 基准：

$$y = w^T x = 0.40 \times 1.00 + (-0.80) \times (-0.49) + 1.20 \times 0.25 = 0.40 + 0.392 + 0.30 = 1.092$$

两者相差约 6.99×10^{-4} ，相对误差 6.4×10^{-4} ，完全在 `check_result` 的 2×10^{-3} RMSE 阈值之内。

观察：误差主要来自 `round`。-0.49 量化成 -62（少了一点点），-0.80 量化成 -85（也少了）。这些每步 0.5 以内的量化噪声在反量化后被放大。当点积长度 D 增大时，噪声的期望趋于相互抵消（零均值），但方差仍线性增长，所以相对误差大致 $O(1/\sqrt{D})$ 。

1.7 隐藏层重量化与误差传播

回顾阶段五：gate/up 投影后经过 SwiGLU ($\text{SiLU}(v) \times u$) 得到隐藏向量 h ，再 `requantize` 回 INT8 进入 down 投影。这个 `requantize` 是量化推理误差的主要来源。

直觉 想象一条流水线：INT8 矩阵乘 $\rightarrow$ FP32 非线性 $\rightarrow$ INT8 矩阵乘 $\rightarrow$ FP32 非线性。每个“ $\rightarrow$ INT8”的环节都是一个有损压缩，会引入 `round` 噪声。而 FP32 非线性（SiLU）对输入微小变化不敏感（导数有界），所以误差不会指数爆炸，但会累积。Lab2 的正确性判定因此采用相对 L2 误差而非逐元素比对：ulp 级 FP32 差异落在 `round` 边界上时，INT8 会差 ± 1 ，再经 down 投影摊到整行。

具体地，`check_result` 检查两个指标：

1. 每个 token 的相对 L2 误差 $< 2 \times 10^{-2}$
2. 全局相对 RMSE $< 2 \times 10^{-3}$

这意味着你的优化实现只需在数值上“接近”参考实现，不要求逐元素一致。这给了你重新排列计算顺序（如改变累加顺序、用 FMA 指令）的自由度。

1.8 思考题 1：算术强度分析

题目：以场景 S3 ($N = 128, D = 256, H = 128, E = 16, K = 4$) 为例，估算参考实现一次前向的总访存量（专家权重被读了多少遍？）和总乘加次数，计算算术强度（MACs/byte）。按专家分组之后这两个数字分别变成多少？由此说明这个负载是访存瓶颈还是计算瓶颈，以及分组为什么能加速。

1.8.1 乘加次数估算

参考实现逐 token 计算。对每个 token：

- 共享专家：gate + up + down 三个矩阵乘， $\text{MACs} = 3DH = 3 \times 256 \times 128 = 98\,304$

- $K = 4$ 个路由专家：每个 $3DH = 98\,304$ ，共 $4 \times 98\,304 = 393\,216$
- 路由打分： $E \times D = 16 \times 256 = 4096$ MACs（相对很小）

单个 token 的专家 MACs $\approx (1 + K) \times 3DH = 491\,520$ 。 $N = 128$ 个 token，总 MACs $\approx 128 \times 491\,520 \approx 6.29 \times 10^7$ 。

1.8.2 访存量估算（按 token 遍历）

参考实现逐 token 遍历。关键观察：**每个 token 都要完整读一遍它所选专家的全部权重。**

数据	单 token 读取	形状	字节数
激活 x_t	1 次	D FP32	1 024 B
路由权重	1 次	$E \times D$ FP32	16 384 B
共享专家权重	1 次	$3DH$ INT8	98 304 B
路由专家权重	K 个专家 $\times 3DH$	$K \times 3DH$ INT8	393 216 B

Table 2: 单 token 读取的数据量（S3 场景）

单个 token 读取的专家权重字节： $(1 + K) \times 3DH = 5 \times 98\,304 = 491\,520$ 字节 ≈ 0.47 MiB。

$N = 128$ 个 token，每个 token 独立遍历自己选的专家。最坏情况下，128 个 token 选的专家互不重叠，总读取量 $\approx 128 \times 491\,520 \approx 63$ MiB。但实际上 S3 只有 $E = 16$ 个路由专家， $K = 4$ ，平均每个路由专家被 $128 \times 4/16 = 32$ 个 token 选中。所以同一份专家权重被反复从内存读入，参考实现没有利用这种复用。

总访存量（不含激活和路由的小头） $\approx N \times (1 + K) \times 3DH \approx 6.29 \times 10^7$ 字节 ≈ 60 MiB。

1.8.3 算术强度

算术强度 = 乘加次数 / 访存字节数：

$$AI_{\text{per-token}} = \frac{491\,520 \text{ MACs}}{491\,520 \text{ bytes}} \approx 1.0 \text{ MAC/byte.} \quad (1.16)$$

直觉 1 MAC/byte 是什么概念？现代 CPU 的 INT8 计算峰值可达数百 GFLOPS，但 DRAM 带宽只有几十 GB/s。算术强度只有 1 意味着每搬 1 字节数据只做 1 次乘加，计算单元大部分时间在等数据。这就是典型的**访存瓶颈**（memory-bound）。Intel Sapphire Rapids 的平衡点（算术强度阈值）大约在几十到上百 MAC/byte，我们离它差了两个数量级。

1.8.4 按专家分组后

如果改成**按专家分组**：先把所有 token 的激活量化好，再对每个专家，一次性处理所有选中它的 token。此时每个专家的权重只从内存读一遍：

- 总访存量（专家权重）： $E \times 3DH = 16 \times 98\,304 = 1\,572\,864$ 字节 ≈ 1.5 MiB（加上共享专家 0.094 MiB）
- 总 MACs 不变： $\approx 6.29 \times 10^7$

新算术强度：

$$AI_{\text{grouped}} = \frac{6.29 \times 10^7}{1.66 \times 10^6} \approx 37.9 \text{ MAC/byte.} \quad (1.17)$$

从 1.0 提升到 37.9，算术强度提升了约 38 倍。这正是分组能显著加速的根本原因：它把“每 token 读一遍权重”变成“每专家读一遍权重”，消除了权重的重复读取，让负载从深度访存瓶颈向计算瓶颈侧移动。

严格说，激活 x_q 现在要被读 K 次（每个被选中的专家读一遍），但 x_q 的体量（ $N \times D = 128 \times 256 = 32\,768$ 字节 INT8）远小于专家权重，且容易驻在 L1/L2 缓存。所以分组的净收益是正的。

1.9 思考题 3：INT8 累加溢出分析

题目：单个 INT8 $\times$ INT8 乘积的最大绝对值是多少？为什么点积必须在 INT32 中累加，若改用 INT16，最坏情况下累加到第几项就会溢出？用允许的最大归约长度（MAX_D_MODEL = 1024）估算最坏情况下的累加值，并说明它离 INT32 的表示上限还有多少余量。

1.9.1 单个乘积的最大值

INT8 的范围是 $[-128, 127]$ 。单个乘积 $w_{q[i]} \times x_{q[i]}$ 的最大绝对值：

$$\text{abs}(w_{q[i]} \times x_{q[i]})_{\max} = 128 \times 127 = 16\,256. \quad (1.18)$$

注意 $(-128) \times (-128) = 16\,384$ 在数学上更大，但 INT8 最小值是 -128 、最大值是 127 ，所以实际最大绝对值是 $(-128) \times 127 = -16\,256$ ，绝对值为 $16\,256$ 。

1.9.2 INT16 何时溢出

INT16 的范围是 $[-32\,768, 32\,767]$ 。假设每个乘积都是最坏值 $16\,256$ ，累加 n 项后达到 $n \times 16\,256$ 。令其 $> 32\,767$ ：

$$n > 32\,767 / 16\,256 \approx 2.006. \quad (1.19)$$

也就是说，最坏情况下累加到第 3 项就会溢出 INT16。即使乘积不是每次都取到最大，只要 D 较大，统计上也很容易突破 INT16。

直觉 这就是为什么硬件提供 INT8 $\times$ INT8 $\rightarrow$ INT16 的乘法指令后，还要提供 INT16 $\rightarrow$ INT32 的累加指令（或直接 INT8 $\times$ INT8 $\rightarrow$ INT32 的 VNNI 指令）。Lab2 的归约长度 D 最大可达 1024，必须用 INT32 累加。

1.9.3 INT32 的余量

INT32 的范围是 $[-2\,147\,483\,648, 2\,147\,483\,647]$ 。最坏情况下， $D = 1024$ 个全为 $16\,256$ 的乘积相加：

$$a_{\max} = 1024 \times 16\,256 = 16\,695\,296 \approx 1.67 \times 10^7. \quad (1.20)$$

INT32 上限 $\approx 2.15 \times 10^9$ ，余量倍数：

$$\frac{2.15 \times 10^9}{1.67 \times 10^7} \approx 128.7. \quad (1.21)$$

例 最坏情况下， $D = 1024$ 的全满量程累加结果约 1.67×10^7 ，距 INT32 上限还有约 128 倍余量。这意味着即使算上多通道并行（如 AMX 的 tile 累加多次）也绰绰有余。但要注意：如果同一累加器被复用做多次点积（例如 GEMM 中 $M \times N$ 输出 tile 共享一个 K 方向累加器），必须确认总累加次数仍在余量内。

实战提示：本实验的 D 和 H 都是 64 的倍数（见 *moe.h* 的校验），这是为了对齐 SIMD lane 和 AMX tile 的边界。但累加长度在运行时变化，你的 INT32 累加器必须按 $\text{MAX_D_MODEL} / \text{MAX_D_FF}$ 上界分配空间，不能假设固定长度。

1.10 SIMD/VNNI/AMX 指令原理

1.11 引言：从自动向量化到手写 SIMD

上一篇教程我们看到，baseline 的内积循环在 -O3 下大部分没有被向量化，少量被向量化的也只用了 128 位 SSE。编译器自动生成的汇编用一串 `pcmpgtb + punpcklbw + pmullw + paddb` 来处理 $\text{INT8} \times \text{INT8} \rightarrow \text{INT32}$ ，效率低下。本章我们亲手写 AVX-512/VNNI 的 intrinsic 代码，用一条 `vpdpbusd` 或 `vpdpwssd` 替代那一整串指令。

读完本章，你将能用 intrinsic 写出一个正确的 INT8 点积，并在汇编层面验证它确实生成了 VNNI 指令。

1.12 AVX 寄存器与数据类型

1.12.1 寄存器层级

x86 SIMD 指令集经历了三代演进，寄存器宽度逐代翻倍：

指令集	寄存器	宽度	INT8 容量	INT32 容量
SSE2	XMM	128 位	16 个	4 个
AVX2	YMM	256 位	32 个	8 个
AVX-512	ZMM	512 位	64 个	16 个

Table 3: SIMD 寄存器层级

AVX-512 还引入了 8 个掩码寄存器（k0 到 k7），每个掩码寄存器的每一位控制一个 lane 是否执行操作，这使得条件操作可以在不破坏向量性的前提下完成。

1.12.2 C++ 数据类型

编译器通过 intrinsic 头文件 `<immintrin.h>` 提供了与寄存器对应的 C++ 类型：

类型	宽度	含义
<code>__m128i</code>	128 位	整数向量（XMM），可放 16 个 INT8 或 4 个 INT32
<code>__m256i</code>	256 位	整数向量（YMM），可放 32 个 INT8 或 8 个 INT32
<code>__m512i</code>	512 位	整数向量（ZMM），可放 64 个 INT8 或 16 个 INT32
<code>__m512</code>	512 位	FP32 向量（ZMM），可放 16 个 float
<code>__mmask16</code>	16 位	掩码寄存器，控制 16 个 lane

Table 4: 常用 intrinsic 数据类型

这些类型在 `<immintrin.h>` 中定义为 `union/struct`，编译器把它们直接映射到寄存器。注意 `__m512i` 中的 `i` 表示整数 (*integer*)，不带后缀的 `__m512` 是浮点。所有整数向量类型不区分元素大小 (`INT8/INT16/INT32` 共用 `__m256i`)，元素大小由具体指令决定。

1.12.3 intrinsic 命名规则

`intrinsic` 函数名遵循统一的命名模式，理解了规则就能“望文生义”：

```
_mm512_dpbusd_epi32
|           |           |
|           |           |--- epi32: packed int32 (32-bit lanes)
|           |--- dpbusd: dot product, byte, unsigned×signed, d (accumulate)
|--- mm512: 512-bit (ZMM)
```

拆解来看：

1. `_mm / _mm256 / _mm512`：操作宽度 (128 / 256 / 512 位)
2. 操作名：如 `loadu` (未对齐加载)、`storeu` (未对齐存储)、`dpbusd` (点积)、`cvtepi8_epi16` (`int8` 扩展到 `int16`)
3. `_epi8 / _epi16 / _epi32`：lane 宽度 (8/16/32 位整数)
4. `_ps / _pd`：packed single (FP32) / packed double (FP64)

直觉 看到 `_mm256_dpbusd_epi32` 就能拆解为：256 位操作、`dpbusd` (`uint8 × int8 → int32` 点积累加)、输出为 packed `int32`。看到 `_mm512_cvtepi8_epi16` 就知道是 512 位的 `int8` 扩展到 `int16`。

1.13 VNNI 指令详解

VNNI (Vector Neural Network Instructions) 是 Intel 为深度学习推理设计的指令扩展，专门加速量化矩阵乘。核心是两条点积指令，它们把“sign extension + 乘法 + 累加”压成一条指令。

1.13.1 vdpwssd: $\text{int16} \times \text{int16} \rightarrow \text{int32}$

```
_mm256_dpwssd_epi32(src, a, b)
```

语义：把两组 256 位 `int16` 向量做乘法，每 2 对乘积累加为 1 个 `int32`，再累加到 `src`。

输入 a (16 个 `int16`)： $a_0 a_1 a_2 a_3 a_4 a_5 a_6 a_7 a_8 a_9 a_{10} a_{11} a_{12} a_{13} a_{14} a_{15}$
 输入 b (16 个 `int16`)： $b_0 b_1 b_2 b_3 b_4 b_5 b_6 b_7 b_8 b_9 b_{10} b_{11} b_{12} b_{13} b_{14} b_{15}$
 → 每 2 对一组：
 $\text{out}[0] = a_0 b_0 + a_1 b_1$
 $\text{out}[1] = a_2 b_2 + a_3 b_3$
 ...
 $\text{out}[7] = a_{14} b_{14} + a_{15} b_{15}$
 再各自累加到 `src` 的 8 个 `int32 lane`

Figure 2: `vdpwssd` 的数据流 (每 2 对 `int16` → 1 个 `int32`)

关键：每 2 对 int16 产生 1 个 int32，不是 4 对。这意味着如果你的点积长度是 D ，一次 vpdwssd 处理 $2 \times 8 = 16$ 个元素（256 位），产生 8 个部分和，还需要把这 8 个部分和再水平相加。

1.13.2 vdpbusd: uint8 $\times$ int8 $\rightarrow$ int32

```
_mm256_dpbussd_epi32(src, a, b)
```

语义：a 是 unsigned int8 (uint8)，b 是 signed int8 (int8)，每 4 对乘积累加为 1 个 int32，再累加到 src。

输入 a (32 个 uint8): $a_0 a_1 a_2 a_3 a_4 a_5 a_6 a_7 \dots$
 输入 b (32 个 int8): $b_0 b_1 b_2 b_3 b_4 b_5 b_6 b_7 \dots$
 $\rightarrow$ 每 4 对一组:
 $\text{out}[0] = a_0 b_0 + a_1 b_1 + a_2 b_2 + a_3 b_3$
 $\text{out}[1] = a_4 b_4 + a_5 b_5 + a_6 b_6 + a_7 b_7$
 $\dots$
 $\text{out}[7] = a_{28} b_{28} + a_{29} b_{29} + a_{30} b_{30} + a_{31} b_{31}$
 再各自累加到 src 的 8 个 int32 lane

Figure 3: vdpbusd 的数据流（每 4 对 byte $\rightarrow$ 1 个 int32）

关键：参数顺序是 (src, unsigned_a, signed_b)。第一个操作数是无符号的，第二个是有符号的。一次 vdpbusd 处理 $4 \times 8 = 32$ 个元素（256 位），产生 8 个部分和。

VNNI 没有 signed int8 $\times$ signed int8 $\rightarrow$ int32 的指令。vdpbusd 要求一个操作数是无符号的。而 Lab2 中权重和激活都是有符号 int8。这个问题有两种解法，下面逐一展开。而 AMX 的 _tile_dpbssd 直接支持 signed $\times$ signed，这是 T5 的内容。

1.14 signed $\times$ signed int8 的两种方案

1.14.1 方案 A: int8 扩展到 int16 + vpdwssd

最直观的方案：用 _mm256_cvtepi8_epi16 把 int8 符号扩展到 int16，再用 vpdwssd 做 int16 $\times$ int16 $\rightarrow$ int32。

直觉 这相当于在数据进入 VNNI 之前先“升级”到更大的类型。好处是逻辑简单，权重和激活都保持有符号。代价是多了一步 sign extension（虽然也是向量化的），而且 int16 $\times$ int16 的吞吐量只有 int8 $\times$ int8 的一半（因为同样的寄存器宽度能放更少的 int16）。

1.14.2 方案 B: 偏移技巧 + vdpbusd

更高效的方案：把激活从 $[-128, 127]$ 偏移到 $[0, 255]$ （加 128），变成 uint8，然后用 vdpbusd。

数学推导如下。设 w_q 为 int8 权重， x_q 为 int8 激活，令 $x_u = x_q + 128$ (uint8)：

$$\sum_i w_{q[i]} \cdot x_{q[i]} = \sum_i w_{q[i]} \cdot (x_{u[i]} - 128) = \underbrace{\sum_i w_{q[i]} \cdot x_{u[i]}}_{\text{vdpbusd 计算}} - 128 \underbrace{\sum_i w_{q[i]}}_{\text{可预计算}} \quad (1.22)$$

因此只需在 preprocess 阶段预计算每行权重的和 $\sum_i w_{q[i]}$ (记为 w_{rowsum})，运行时做一次 vpdpbussd，再减去 128 倍 w_{rowsum} 即可。

例 以 T1 中的三维点积为例。 $w_q = [42, -85, 127]$, $x_q = [127, -62, 32]$ 。

偏移后 $x_u = [255, 66, 160]$, $w_{\text{rowsum}} = 42 + (-85) + 127 = 84$ 。

vpdpbusd 计算 $42 \cdot 255 + (-85) \cdot 66 + 127 \cdot 160 = 10710 - 5610 + 20320 = 25420$ 。

补偿: $25420 - 128 \times 84 = 25420 - 10752 = 14668$ 。

与 T1 中手算的 INT32 结果 14 668 完全一致。

方案 B 的优势: 吞吐量翻倍 (int8 比 int16 多一倍)，且 vpdpbussd 一步到位，无需 sign extension 步骤。代价是需要预计算 w_{rowsum} 并在结果中减去补偿项。

1.15 AMX 编程模型与 Tile 寄存器

1.16 引言：从 VNNI 到 AMX

T3 中我们用 VNNI 的 vpdpbussd 做向量点积，每条指令处理 32 个 INT8 元素。但 VNNI 仍是“向量乘向量”的模式，每次产生一个部分和，还需要水平求和才能得到标量结果。

AMX (Advanced Matrix Extensions) 是 Intel 在 Sapphire Rapids 中引入的矩阵加速单元，它把计算模型从“向量乘向量”升级为“矩阵乘矩阵”。一条 AMX 指令可以完成一个 $16 \times K$ 的 INT8 矩阵乘以 $K \times 16$ 的 INT8 矩阵，直接产生 16×16 的 INT32 结果。更关键的是，AMX 的 `_tile_dpbssd` 直接支持 signed INT8 $\times$ signed INT8，不需要 T3 中的偏移技巧。

AMX 只在 *Sapphire Rapids* 及更新的 *Intel CPU* 上可用。本地 *Meteor Lake* 不支持 AMX，所以本章的代码只能在集群上编译运行。但理解 AMX 的编程模型对 *Lab2* 冲击高分至关重要。

1.17 AMX 编程模型

1.17.1 Tile 寄存器

AMX 引入了 8 个 **tile 寄存器** (TMM0 到 TMM7)，每个 tile 是一个 2D 矩阵。与 SIMD 的 1D 向量寄存器不同，tile 有行和列两个维度：

属性	INT8 tile	INT32 tile (累加器)	说明
最大行数	16	16	固定上限
最大列数	1024 字节	1024 字节	实际列数取决于元素大小
INT8 列数	最多 1024	N/A	8 位元素
INT32 列数	N/A	最多 256	32 位元素
寄存器数量	8 个	8 个	共享 TMM0-TMM7

Table 5: AMX tile 寄存器属性

直觉 把 tile 想象成一块二维“画布”。`_tile_loadd` 从内存加载一块 INT8 数据到画布上，`_tile_dpbssd` 把两块 INT8 画布做矩阵乘法，结果累加到第三块 INT32 画布上，

`_tile_stored` 把结果画布存回内存。8 块画布可以复用，比如 2 块用于输入、1 块用于累加。

1.17.2 Tile 配置 (Palette)

使用 AMX 前，必须先配置 tile 的形状。这个过程叫 **tile configuration**，通过 **palette** 选择预配置。

```
#include <immintrin.h>

// 配置 AMX tile
struct __tile_config {
    uint8_t palette_id;        // 0 = 关闭, 1 = INT8/INT16/BF16
    uint8_t start_row;         // 通常为 0
    uint8_t reserved_0[14];
    uint16_t tile_rows[8];     // 每个 tile 的行数 (1-16)
    uint16_t tile_cols[8];     // 每个 tile 的列字节数 (需为 64 的倍数)
    uint8_t reserved_1[16];
};

void init_amx() {
    __tile_config cfg = {};
    cfg.palette_id = 1;
    for (int i = 0; i < 8; i++) {
        cfg.tile_rows[i] = 16;    // 16 行
        cfg.tile_cols[i] = 64;    // 64 字节 = 64 个 INT8 或 16 个 INT32
    }
    _tile_loadconfig(&cfg);
}
```

配置好后，每个 tile 是 16×64 字节的 2D 矩阵。对于 INT8，这是 16×64 个元素；对于 INT32，这是 16×16 个元素。

`tile_cols` 的单位是字节，不是元素个数。64 字节的 INT8 tile 有 64 列，64 字节的 INT32 tile 有 16 列。`_tile_dpbssd` 要求两个 INT8 输入 tile 的列数之和等于 INT32 输出 tile 的列数的 4 倍（因为 4 个 INT8 乘积累加为 1 个 INT32）。

1.18 核心 AMX 指令

1.18.1 加载: `_tile_loadd`

```
_tile_loadd(int tile_idx, const void* base, int stride);
```

从 base 地址加载一个 2D tile，行间距由 stride 指定。例如加载 16×64 字节的 INT8 矩阵，stride 就是矩阵的行宽（以字节为单位）。

1.18.2 矩阵乘加: `_tile_dpbssd`

```
_tile_dpbssd(int dst, int a, int b);
```

计算 $C += A \times B$, 其中 A 和 B 是 signed INT8 tile, C 是 INT32 tile。这正好是 Lab2 需要的 signed $\times$ signed INT8 点积。

A tile (INT8, $16 \times K$) $\times$ B tile (INT8, $K \times N$) $\rightarrow$ C tile (INT32, $16 \times N$)

其中 K 由 tile 的列数决定 (INT8 列数 = K)。

N 由 INT32 输出 tile 的列数决定 ($N = K/4$)。

典型配置: $K = 64$, $N = 16$ 。

一条 `_tile_dpbssd` 完成 $16 \times 64 \times 16 = 16\,384$ 个 INT8 乘加。

Listing 1: `_tile_dpbssd` 的矩阵乘语义

直觉 对比 VNNI 的 `vpdpbusd`: 一条 256 位 VNNI 指令做 $4 \times 8 = 32$ 个乘加; 一条 AMX `_tile_dpbssd` 做 $16 \times 64 \times 16 = 16\,384$ 个乘加。AMX 的吞吐量是 VNNI 的约 512 倍。当然实际性能受限于 tile 加载和内存带宽, 但这个量级差距解释了为什么 Lab2 推荐用 AMX 追求高分。

1.18.3 存储: `_tile_stored`

```
_tile_stored(int tile_idx, void* base, int stride);
```

把 tile 内容存回内存, 行间距由 `stride` 指定。

1.18.4 清零: `_tile_zero`

```
_tile_zero(int tile_idx);
```

把指定 tile 清零, 在累加前初始化输出 tile。

1.19 Top-down 与 Roofline 模型

1.20 引言: 从“哪里慢”到“为什么慢”

T6 中的 Hotspots 分析告诉你“时间花在了哪个函数”, 但没告诉你“为什么这个函数慢”。是计算单元不够用? 是等内存? 还是前端取指跟不上? 本章介绍两种更深层的分析方法: **Top-down 微架构分析**解释流水线停顿的原因, **Roofline 模型**把程序性能与硬件上限放在同一张图中比较。

1.21 CPU 流水线模型

要理解 Top-down 分析, 先要有一个现代 CPU 流水线的粗略模型。把流水线分成**前端**和**后端**两段:

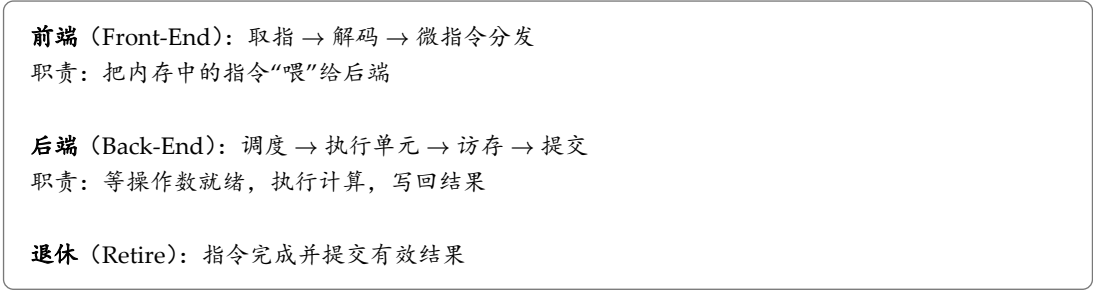


Figure 4: CPU 流水线的前端/后端模型

直觉 前端像一个厨房的传菜员, 负责把菜单(指令)从仓库(内存)取来分发给厨师。后端是厨师团队, 等食材(操作数)备齐后开火做菜(执行计算)。如果传菜员太慢, 厨师闲着等菜单(Front-End Bound)。如果厨师在做菜, 但食材还没到(等内存), 也是闲着(Back-End Bound / Memory Bound)。只有厨师一直在做菜且不返工, 才是高效(Retiring 高)。

1.22 Top-down 四分类

Top-down 分析把流水线中的每个周期归为四类之一:

分类	含义	Lab2 中的常见原因
Front-End Bound	前端供给指令的速度跟不上	指令缓存未命中、解码瓶颈
Bad Speculation	分支预测错误导致已执行工作被丢弃	Top-K 选择中的分支
Back-End Bound	执行资源忙碌或数据尚未到达	见下文细分
Retiring	指令完成并提交有效结果	占比高通常是好现象

Table 6: Top-down 四分类

Retiring 高不等于已经达到峰值。比如标量代码的 Retiring 可能很高(每条指令都在做有用的事), 但用 VNNI 一条指令替代十条标量指令后, Retiring 可能降低(因为等待 VNNI 指令的结果), 但实际性能更高。所以 Top-down 分析要结合绝对性能指标一起看。

1.23 Back-End Bound 细分

Back-End Bound 是 Lab2 中最常见的瓶颈, 需要进一步区分:

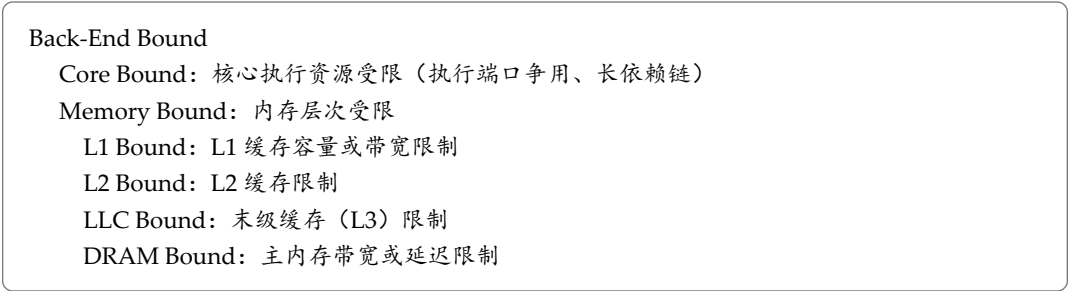


Figure 5: Back-End Bound 的层级细分

直觉 Memory Bound 告诉你“在等内存”，而 L1/L2/LLC/DRAM 告诉你“在等哪一级内存”。这对选择优化策略至关重要：如果 L1 Bound，说明数据在缓存中但带宽不够用，可以尝试减少数据搬运量（如寄存器分块）；如果 DRAM Bound，说明数据根本不在缓存中，需要减少重复读取（如按专家分组）。

1.23.1 Lab2 中的典型 Top-down 结果

对 baseline 做 Microarchitecture Exploration，预期看到：

指标	Baseline 典型值	含义
Retiring	$\approx 20 - 30\%$	有效指令占比低，大量周期在等
Front-End Bound	$\approx 5 - 10\%$	前端基本不是瓶颈
Bad Speculation	$\approx 5 - 10\%$	分支预测错误较少
Back-End Bound	$\approx 50 - 70\%$	主要瓶颈：等数据或等执行
Back-End $\rightarrow$ Memory Bound	$\approx 40 - 50\%$	内存是主要瓶颈

Table 7: Baseline 典型 Top-down 结果（预期）

这与 T1 中算术强度只有约 1 MAC/byte 的分析一致：Back-End Bound 中的 Memory Bound 占大头，说明 CPU 在等数据从内存到达。

1.23.2 优化后的预期变化

1. **分组后**：Memory Bound 从 DRAM Bound 降到 L2 Bound（权重驻留 L2），Back-End Bound 降低
2. **VNNI 后**：Retiring 升高（一条指令做更多有效工作），Core Bound 可能上升（VNNI 执行单元繁忙）
3. **AMX 后**：Retiring 进一步升高，计算密度大幅增加，可能从 Memory Bound 转向 Core Bound

Top-down 的价值在于“可解释的演进”：每做一次优化，你能看到哪个指标改善了，哪个指标变差了。比如分组后 *Memory Bound* 降低但 *Bad Speculation* 可能上升（分组引入了分支），这种权衡只有 *Top-down* 才能看到。

1.24 Roofline 模型

1.24.1 基本概念

Roofline（屋顶线）模型把程序的性能与硬件上限放在同一张图中。横轴是**算术强度**（Arithmetic Intensity, AI，单位 MAC/byte），纵轴是**性能**（FLOPS 或 MAC/s）：

$$\text{Performance} = \min(\text{AI} \times \text{Bandwidth}, \text{Peak FLOPS}) \quad (1.23)$$

图中有一条斜线（带宽限制，斜率 = 带宽）和一条水平线（计算峰值上限）。程序点落在斜线下方表示**访存瓶颈**，落在水平线下方且靠近水平线表示**计算瓶颈**。

直觉 把 Roofline 想象成一栋房子的屋顶：左边是斜坡（带宽限制），越往右越接近天花板（计算峰值）。程序点在斜坡上时，增加算术强度能让它“爬得更高”（更快）；一旦上了天花板，再增加算术强度也没用，需要更强的计算单元（如 VNNI/AMX）。

1.24.2 Lab2 的 Roofline 分析

以 S3 场景 ($N = 128, D = 256, H = 128, E = 16, K = 4$) 为例，用 T1 中的估算：

版本	AI (MAC/byte)	瓶颈位置	优化方向
Baseline (逐 token)	≈ 1	斜坡底部 (DRAM)	分组
分组后	≈ 38	斜坡中部 (L2)	VNNI/AMX
VNNI 后	≈ 38	接近天花板	AMX
AMX 后	$\approx 38 +$	天花板	减小 K 循环开销

Table 8: S3 场景各版本的 Roofline 预期

算术强度 38 MAC/byte 在 *Sapphire Rapids* 上可能仍在斜坡和天花板的交界处。具体是访存瓶颈还是计算瓶颈，取决于硬件的 L2 带宽和 INT8 计算峰值的比值。实际判断需要用 VTune 的 *Memory Access* 分析获取缓存命中率和带宽利用数据。

1.24.3 估算运算量与数据搬运量

要画 Roofline 图，需要估算两个量：

1. **运算量** (MAC 数)： $N \times (K + 1) \times 3DH$ 。S3 场景为 $128 \times 5 \times 98\,304 \approx 6.29 \times 10^7$ MAC
2. **数据搬运量** (字节)： 分组后 $\approx (E + 1) \times 3DH$ 字节（权重只读一遍） + $N \times D$ 字节（激活） $\approx 1.67 \text{ MiB} + 0.03 \text{ MiB} \approx 1.7 \text{ MiB}$

算术强度 = $6.29 \times 10^7 / 1.78 \times 10^6 \approx 35 \text{ MAC/byte}$ 。

1.25 IPC 与其他指标

Top-down 分析中，还有几个值得记录的辅助指标：

指标	含义	理想值
IPC	每周期退休指令数	> 3 表示高效， < 1 表示停顿多
频率	CPU 运行频率	注意 AVX-512 可能降频
L2 命中率	L2 缓存命中比例	分组后应 $> 90\%$
LLC 命中率	L3 缓存命中比例	取决于工作集大小
DRAM 带宽利用	实际 / 峰值带宽	低表示未受 DRAM 限制

Table 9: 辅助性能指标

直觉 IPC 是一个快速的“健康度”指标。Baseline 的 IPC 可能只有 0.5-1.0（大量停顿等内存），优化后 IPC 升高（更多周期在做事）。但 IPC 高不一定等于性能好，还要看每条指令做了多少有效工作（CPI 与 Retiring 结合看）。

1.26 性能上限分析

优化后期，建议为热点建立一份简要的性能上限分析，判断还有多少优化空间。

例 以 S3 场景为例，假设优化后的性能为 P (MAC/s)，Roofline 天花板为 P_{peak} (INT8 计算峰值)。

1. 估算运算量 6.29×10^7 MAC，数据搬运量 1.78×10^6 字节， $\text{AI} \approx 35$ MAC/byte
2. 查 Sapphire Rapids 的 INT8 峰值（如 $2 \times 16 \times 16 \times 64 = 32\,768$ MAC/cycle/core，按 3 GHz 约 98 GMAC/s/core）
3. 查 L2 带宽上限（约 100 GB/s/core）
4. Roofline 预测的 AI 交叉点： $35 \times 100 \times 10^9 \approx 3.5$ TMAC/s，远超计算峰值，说明 35 MAC/byte 已经在计算天花板附近
5. 比较 P 与 P_{peak} ：如果 $P/P_{\text{peak}} > 80\%$ ，说明已经很接近上限，剩余空间有限

实际的性能上限分析要复杂得多：多线程下的带宽竞争、AVX-512 降频、AMX tile 加载开销等都会影响。但这个“粗估”已经能帮你判断“还有没有优化空间”，避免在不值得的地方投入时间。

2 Part II: GPU 侧原理

2.1 Linear Attention 与 Gated DeltaNet

2.2 引言：为什么需要 Linear Attention

标准 Transformer 的核心是**注意力机制** (Attention)。它让模型在处理当前 token 时，能够“回看”前面所有 token，决定该关注谁。但标准注意力有一个绕不开的代价：序列长度一旦变长，计算量和内存就平方级膨胀。当序列长度 L 从 4096 涨到 32768，注意力的计算量不是变 8 倍，而是变 64 倍。

这个平方复杂度的根源在于构造 QK^T 这个 $L \times L$ 的中间矩阵。**Linear Attention**（线性注意力）的核心思想是：能否绕过这个 $L \times L$ 矩阵，直接把复杂度降到线性？

本教程是 Lab3 系列的第一篇。你读完后应该能：写出标准注意力的代价分析、推导 Linear Attention 的递推形式、理解 GLA / DeltaNet / Gated DeltaNet 三种改进各自解决什么问题，并为后续的 chunk-wise parallel 推导和 TileLang 实现建立清晰的算法心智模型。

直觉 不妨把注意力机制想象成一个人在开会时做笔记。标准注意力相当于把每个人的发言都和别人一一对照记下来，人数一多笔记就爆炸；Linear Attention 相当于维护一本“会议摘要本”，每来一条新发言就更新摘要，查询时只查摘要本，摘要本的大小和会议长短无关。

2.3 标准注意力的代价分析

标准的因果自注意力可以写为：

$$O = \text{softmax}\left(\frac{QK^T + M}{\sqrt{d_k}}\right)V. \quad (2.1)$$

其中 $Q, K \in \mathbb{R}^{L \times d_k}$, $V \in \mathbb{R}^{L \times d_v}$, M 是下三角的因果掩码（保证只看过去）。这个公式里最致命的一步是 QK^T ，它产生一个 $L \times L$ 的矩阵。

我们来算一笔账。设序列长度 L 、head dimension $d_k = d_v = 128$ （本实验的设定）。

$$\text{计算量} \approx 2L^2 d_k \quad \text{访存量} \approx 2L^2 \text{ (attention 矩阵)} + (QKV) \approx 2Ld. \quad (2.2)$$

注意 attention 矩阵 $L \times L$ 在 softmax 时需要逐行读写，这是主要访存开销。 $L = 32768$ 时，一个 attention 矩阵在 FP32 下是 4 GB，远超 GPU 片上缓存。

当 L 较小（如 4096）时，GPU 的算力足以压住访存；但当 L 增长到 32768 以上， L^2 项迅速膨胀，成为主要瓶颈。这就是 Linear Attention 的动机：把 L^2 降到 L 。

2.4 Linear Attention：用状态递推替代 $L \times L$ 矩阵

2.4.1 从 softmax 到特征映射

标准注意力用 $\exp(\text{cdot})$ 作为相似度的核函数。Linear Attention 的思路是用一个特征映射 $\varphi(\text{cdot})$ 近似它：

$$\exp(QK^T) \approx \varphi(Q)\varphi(K)^T. \quad (2.3)$$

这样原本的“先算 QK^T 再乘 V ”，就可以利用矩阵乘法结合律改写。对第 t 个 token 的输出：

$$o_t = \frac{\sum_{i=1}^t \varphi(q_t) \varphi(k_i)^T v_i}{\sum_{i=1}^t \varphi(q_t) \varphi(k_i)^T}. \quad (2.4)$$

由于 $\varphi(q_t)$ 与求和无关，可以提出求和：

$$o_t = \frac{\varphi(q_t) \left(\sum_{i=1}^t \varphi(k_i)^T v_i \right)}{\varphi(q_t) \left(\sum_{i=1}^t \varphi(k_i)^T \right)}. \quad (2.5)$$

直觉 这里的“魔法”就是结合律： $(QK^T)V = Q(K^TV)$ 。原本要先算 $L \times L$ 再算 $L \times d_v$ ，现在先算 $d_k \times d_v$ 的 K^TV ，再算 $L \times d_v$ 。 K^TV 的大小和 L 无关！

2.4.2 状态递推形式

定义状态 $S_t = \sum_{i=1}^t k_i^T v_i \in \mathbb{R}^{d_k \times d_v}$ ，则输出简化为 $o_t = q_t S_t$ （省略归一化）。关键在于 S_t 可以递推：

$$S_t = S_{t-1} + k_t^T v_t, \quad o_t = q_t S_t. \quad (2.6)$$

例 取 $d_k = d_v = 2$ ，三个 token： $k_1 = (1, 0)$, $v_1 = (3, 0)$, $k_2 = (0, 1)$, $v_2 = (0, 5)$, $q_3 = (2, 1)$ 。

递推过程：

1. $S_1 = k_1^T v_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix} (3, 0) = \begin{pmatrix} 3 & 0 \\ 0 & 0 \end{pmatrix}$
2. $S_2 = S_1 + k_2^T v_2 = \begin{pmatrix} 3 & 0 \\ 0 & 0 \end{pmatrix} + \begin{pmatrix} 0 \\ 1 \end{pmatrix} (0, 5) = \begin{pmatrix} 3 & 0 \\ 0 & 5 \end{pmatrix}$
3. $o_3 = q_3 S_2 = (2, 1) \begin{pmatrix} 3 & 0 \\ 0 & 5 \end{pmatrix} = (6, 5)$

状态 S 始终是 2×2 ，与 token 数无关。

2.4.3 复杂度与局限

递推形式下，每步只更新 $d_k \times d_v$ 的状态，总计算量关于 L 为线性，特别适合 autoregressive decoding。但这也带来一个根本局限：

状态 S_t 的大小与序列长度无关，既是 *Linear Attention* 的优势，也是它的诅咒。所有历史关联都被压缩到同一个矩阵中，旧信息不可避免地互相干扰。

纯加法更新没有主动遗忘或覆盖机制：一旦某个 $k_i^T v_i$ 写入状态，它只能被后续不断累加的新信息稀释，无法被精确修改或删除。后续的 GLA、DeltaNet、Gated DeltaNet 都在改进这个状态更新规则。

2.5 GLA：加入遗忘门

GLA（Gated Linear Attention，门控线性注意力）在状态递推中加入由当前输入决定的遗忘门。用简化的标量门表示，其更新为：

$$S_t = \alpha_t S_{t-1} + k_t^T v_t, \quad \alpha_t \in (0, 1). \quad (2.7)$$

直觉 α_t 就像人脑的遗忘旋钮。接近 1 时记忆基本保留，接近 0 时快速清空。与固定衰减相比，数据相关的 α_t 让模型根据当前输入决定“这一刻该忘多少”。

实际的 GLA 可以使用更细粒度的门（per-channel decay），但核心思想一致。 α_t 缓解了状态无限累积的问题，但写入仍然是简单的外积相加 $k_t^T v_t$ ，无法针对与当前 key 冲突的旧关联做精确修改。

2.6 DeltaNet：delta rule 定向覆盖

DeltaNet 把加法写入替换为 **delta rule**（ δ 规则）。它先用当前 key 从旧状态中读取预测值，再根据预测与目标 value 之间的误差更新状态：

$$\hat{v}_t = k_t S_{t-1}, \quad (2.8)$$

$$S_t = S_{t-1} + \beta_t k_t^T (v_t - \hat{v}_t), \quad \beta_t \in (0, 1). \quad (2.9)$$

直觉 DeltaNet 像是在做“纠错”。它先问“旧的记忆里这个 key 对应什么 value”，得到预测 $\hat{v}_t$ ，然后把预测误差 $(v_t - \hat{v}_t)$ 写回去。如果预测正确，状态不变；如果预测错误，状态被修正。

在 key 已归一化的情况下， β_t 可以理解为写入强度： $\beta_t = 0$ 时不修改状态， $\beta_t = 1$ 时当前 key 对应的旧预测被完整替换为 v_t 。因此 DeltaNet 能够定向修正已有的 key-value 关联。

但 DeltaNet 没有独立的全局遗忘门，难以快速清除大量已经无关的记忆。

例 承接上节的 $S_2 = \begin{pmatrix} 3 & 0 \\ 0 & 5 \end{pmatrix}$ ，现来 $k_3 = (1, 0)$ （与 k_1 同方向）， $v_3 = (10, 0)$ ， $\beta_3 = 1$ 。预测 $\hat{v}_3 = k_3 S_2 = (1, 0) \begin{pmatrix} 3 & 0 \\ 0 & 5 \end{pmatrix} = (3, 0)$ 。

误差 $v_3 - \hat{v}_3 = (10, 0) - (3, 0) = (7, 0)$ 。

更新 $S_3 = S_2 + k_3^T(7, 0) = \begin{pmatrix} 3 & 0 \\ 0 & 5 \end{pmatrix} + \begin{pmatrix} 1 \\ 0 \end{pmatrix}(7, 0) = \begin{pmatrix} 10 & 0 \\ 0 & 5 \end{pmatrix}$ 。

对比 Linear Attention 会得到 $\begin{pmatrix} 13 & 0 \\ 0 & 5 \end{pmatrix}$ (继续累加)。DeltaNet 用新的 $v_3 = 10$ 覆盖了旧值 $v_1 = 3$ ，而不是叠加。

2.7 Gated DeltaNet: 门控遗忘与 delta update 的组合

Gated DeltaNet (GDN) 把门控遗忘与 delta update 组合起来：

$$\bar{S}_t = \alpha_t S_{t-1}, \quad (2.10)$$

$$S_t = \bar{S}_t + \beta_t k_t^T (v_t - k_t \bar{S}_t), \quad (2.11)$$

$$o_t = q_t S_t. \quad (2.12)$$

等价地，状态更新也可以写为：

$$S_t = \alpha_t (I - \beta_t k_t^T k_t) S_{t-1} + \beta_t k_t^T v_t. \quad (2.13)$$

直觉 GDN 是“两手抓”： α_t 负责控制旧状态整体保留多少（快速遗忘）， β_t 负责控制当前 key 对应的关联修改多少（精确覆盖）。两种机制分工不同且互补：门控适合快速释放状态容量，delta rule 适合精确覆盖特定记忆。

模型	状态更新的核心机制	能力与局限
Linear Attention	直接累加 $k_t^T v_t$	简单高效，但旧关联只能累积
GLA	写入前对旧状态施加数据相关 decay	能主动遗忘，但写入仍是加法
DeltaNet	根据预测误差定向修改关联	能精确覆盖，但缺少独立的快速遗忘机制
Gated DeltaNet	decay + delta rule	同时支持快速遗忘与定向更新

Table 10: 四种模型的状态更新机制对照

2.8 计算接口与数据约定

本实验关注的不是单个 token 的 decode，而是 prefill：一次处理整段提示词，生成各层后续 decode 所需的状态。Prefill 提供了序列维度上的并行机会，但 GDN 的状态递推仍引入因果依赖。

实验固定 chunk size $C = 64$ ， $d_k = d_v = 128$ 。函数 `gdn_prefill_forward` 的接口如下：

张量	形状	数据类型	含义
q, k	[B, T, H _q , dk]	BF16	已 L2 归一化的 query / key
v	[B, T, H _v , dv]	BF16	value
g_cumsum	[B, T, H _v]	FP32	chunk 内 log 空间门控前缀和
beta	[B, T, H _v]	FP32	delta rule 的写入强度
A	[B, T, H _v , 64]	BF16	分块 KKT 下三角矩阵的逆
initial_state	[B, H _v , dk, dv]	FP32	可选初始状态
output	[B, T, H _v , dv]	BF16	prefill 输出
final_state	[B, H _v , dk, dv]	FP32	最终状态

Table 11: gdn_prefill_forward 接口定义

注意门控前缀和 g_{cumsum} 是 $\log$ 空间的: $g_{c,r}^{\text{cumsum}} = \sum_{j=1}^r \text{raw_g}_{c,j} = \log \gamma_{c,r}$, 因此 $\gamma_{c,r} = \exp(g_{c,r}^{\text{cumsum}})$ 。在公式中使用 γ 时需对 g_{cumsum} 取 $\exp$ 。

此外还有两种输入变种需要正确处理:

1. 给定 S_0 : 部分 case 传入非零 initial_state, 其余从零状态开始。
2. GVA (Group-Value Attention): 可能 $H_v > H_q$, 且 H_v 能被 H_q 整除。令组大小 $G = \frac{H_v}{H_q}$, 第 h_v 个 value head 使用的 query/key head 为 $h_{\text{qk}} = \lfloor \frac{h_v}{G} \rfloor$ 。

2.9 Chunk-wise Parallel 推导

2.10 引言：在递推与并行之间折中

上一章我们建立了 GDN 的状态递推形式: $S_t = \alpha_t S_{t-1} + \beta_t k_t^T (v_t - k_t \bar{S}_t)$, 其中 $\bar{S}_t = \alpha_t S_{t-1}$ 。这个公式优美且与序列长度线性相关, 但在 prefill 场景下, 它有一个工程上的致命缺陷: 每一步都依赖前一步, GPU 没法把它并行展开。

最朴素的补救办法是回到标准注意力的完全并行形式 $O = (QK^T \odot M)V$, 让整段序列一次性算完。可这一步又把代价推回 $O(L^2d)$, $L \times L$ 的中间矩阵会让显存爆炸。串行太慢, 全并行太贵, 我们卡在中间。

本章我们要推导的 **chunk-wise parallel** (分块并行), 就是走出第三条路: 把序列切成固定大小的 chunk, chunk 内用矩阵乘并行计算, chunk 间用状态递推串行传递。它会同时出现矩阵 A 、中间量 U 与 W 、状态 $S_{[c]}$ 与输出 $O_{[c]}$, 看上去吓人, 但每一步都有清晰的物理意义。读完本章, 你应该能徒手把 chunk-wise 公式从递推形式推出来, 并能解释为什么实验里固定取 $C = 64$ 。

直觉 不妨把递推想象成“一个人逐条读消息做笔记”, 把完全并行想象成“会议室里所有人同时喊出对所有其他人的关注”。chunk-wise 是折中: 每 64 条消息为一组, 组内大家并行讨论得出小组摘要, 然后由一位书记员把摘要传到下一组。这样并行度大幅提升, 又不需要所有人同时喊话。

2.11 递推的并行困境：为什么逐 token 难以并行

我们用 GDN 的递推形式来分析。对第 t 个 token, 状态更新需要:

$$S_t = \alpha_t S_{t-1} + \beta_t k_t^T (v_t - k_t \bar{S}_t), \quad \bar{S}_t = \alpha_t S_{t-1}. \quad (2.14)$$

注意 S_t 同时依赖 S_{t-1} 和 $\bar{S}_t$ ，而后者又依赖 S_{t-1} 。所以本质上：**算 S_t 之前必须先算完 S_{t-1}** 。这是一条长度为 L 的依赖链。

GPU 上的并行单位是 **Warp**（线程束），一个 Warp 里 32 个 Thread 在同一时钟周期执行同一条指令。如果 32 个 Thread 分别负责 32 个 token 的状态更新，它们会因为互相依赖而被迫串行：Thread 0 算完 S_1 后 Thread 1 才能开始算 S_2 ，以此类推。结果是 Warp 里 31 个 Thread 在干等，**并行度退化到 1/32**。

更严格地说，递推形式会让 GPU 的“算术强度”（计算量与访存量之比）极低。每一步只做 $d_k \times d_v$ 的小矩阵更新（本实验 128×128 ），计算量约 $2d_k d_v \approx 3.3 \times 10^4$ FLOPs，而从显存读 S_{t-1} 又要 $d_k d_v \times 4$ 字节（FP32），算术强度仅约 8 FLOPs/Byte。H100 的算术强度门槛在 50 以上才会被算力限制，否则被访存拖死。

更糟的是，递推形式无法利用 **Tensor Core**（张量核心）。Tensor Core 要求一次矩阵乘的形状至少是 m16 n8 k16（BF16），而单 token 的状态更新只有 $d_k \times d_v$ 一个外积加上一个向量矩阵乘，远小于 Tensor Core 的最小粒度。这意味着 GPU 的算力被严重浪费。

完全并行形式 $O = (QK^T \odot M)V$ 看似能解决并行度问题，但它有自己的代价。

2.12 完全并行形式的代价： $L \times L$ 矩阵的诅咒

如果我们忽略 GDN 的特殊更新规则，只把它“摊平”成类似标准注意力的形式，会得到一个 $L \times L$ 的中间矩阵。设 $Q, K \in \mathbb{R}^{L \times d_k}$ ， $V \in \mathbb{R}^{L \times d_v}$ ，输出可以写成：

$$O = (QK^T \odot M)V. \quad (2.15)$$

其中 M 是下三角因果掩码， $\odot$ 是逐元素乘。这一步直接产生了 $QK^T \in \mathbb{R}^{L \times L}$ 这个致命的中间量。

我们来算一笔账。设 $L = 32768$ ， $d_k = d_v = 128$ （本实验设定）。

1. 计算量： $2L^2 d_k \approx 2 \times 32768^2 \times 128 \approx 2.7 \times 10^{11}$ FLOPs，单 head 单 batch。
2. 访存量： QK^T 矩阵本身的读写就达到 $L^2 \times 2 \text{ byte} \approx 2 \text{ GB}$ （BF16），softmax/归一化阶段还要再读写一次。
3. 显存占用： $L \times L$ 的中间矩阵在 FP32 下是 4 GB，远超 SM 私有的 Shared Memory 容量（H100 SMEM 上限 228 KB）。

真实部署里，attention kernel 通常用 **FlashAttention** 类似的 online softmax 把 $L \times L$ 矩阵切分到 SMEM 里。但 GDN 的 delta rule 不是简单的 softmax 归一化，它要在 chunk 内做“纠错”型更新，FlashAttention 的标准重排并不直接适用。这就是为什么需要 *chunk-wise parallel* 这个专门的形式。

所以，完全并行形式虽然 GPU 友好（一次大 GEMM），却把 $O(L)$ 的递推变成 $O(L^2)$ 的计算和访存， L 一大就会击穿显存带宽。我们需要一个既能让 GPU 并行，又不要 $L \times L$ 中间矩阵的方案。

2.13 Chunk-wise 的核心思想：切一刀在中间

chunk-wise parallel（分块并行）的核心思想很简单：把长度 L 的序列切成 L/C 个长度为 C 的 chunk，对每个 chunk 同时做两件事：

1. **chunk 内**：用矩阵乘并行计算 chunk 内 C 个 token 的相互影响。这是一个 $C \times C$ 的中间矩阵，规模可控。
2. **chunk 间**：用状态递推串行传递 chunk 边界状态 $S_{[c]}$ 。chunk 之间只有 L/C 步串行，远少于 L 步。

直觉 把 chunk 想象成“小组会议”。每个小组 C 个人在组内并行讨论（矩阵乘），同时每个小组出一位代表把“组结论摘要” $S_{[c]}$ 传给下一组（递推）。组内并行度仍是 C ，组间串行步数降到 L/C 。只要 C 选得合适，GPU 既能用 Tensor Core 算组内矩阵乘，又能在 L/C 步内串行完成组间传递。

形式化一点，把 chunk c ($c = 0, 1, \dots, L/C - 1$) 内的 K, Q, V 记为 $K_c, Q_c, V_c \in \mathbb{R}^{C \times d}$ ，把 chunk 边界的状态记为 $S_{[c]} \in \mathbb{R}^{d_k \times d_v}$ 。chunk 内 token r ($r = 1, \dots, C$) 的位置状态记为 $S_{c,r}$ ，满足 $S_{c,0} = S_{[c]}$ 与 $S_{c,C} = S_{[c+1]}$ 。我们要做的事就是：

1. 用 K_c, V_c, Q_c 这 C 行数据，并行算出 $S_{c,1}, S_{c,2}, \dots, S_{c,C}$ 的“内部贡献”。
2. 把 $S_{[c]}$ 经过 chunk 内的衰减和更新，得到下一个 chunk 的边界状态 $S_{[c+1]}$ 。
3. 用 Q_c 和“内部贡献”以及 $S_{[c]}$ 共同算出 chunk 内每个 token 的输出 $O_c \in \mathbb{R}^{C \times d_v}$ 。

接下来两节我们分两步推导：先从最简单的 Linear Attention（无门、无 delta）开始，再推广到 GDN。

2.14 从 Linear Attention 到 chunk-wise 公式

2.14.1 Linear Attention 的递推回顾

Linear Attention 的递推形式是 $S_t = S_{t-1} + k_t^T v_t$ ，输出 $o_t = q_t S_t$ 。我们把它对 chunk c 内的 C 个 token 展开：

$$S_{c,r} = S_{c,r-1} + k_{c,r}^T v_{c,r}, \quad r = 1, \dots, C. \quad (2.16)$$

把这 C 步累加起来，chunk 边界状态：

$$S_{[c+1]} = S_{[c]} + \sum_{r=1}^C k_{c,r}^T v_{c,r} = S_{[c]} + K_c^T V_c. \quad (2.17)$$

这一步完全并行： $K_c^T V_c$ 是一个 $d_k \times d_v$ 的小矩阵乘， $K_c \in \mathbb{R}^{C \times d_k}$ ， $V_c \in \mathbb{R}^{C \times d_v}$ ，可以用 Tensor Core 一次算完。

2.14.2 chunk 内输出的拆分

对 chunk 内 token r 的输出 $o_{c,r} = q_{c,r} S_{c,r}$ 。把 $S_{c,r}$ 拆成 $S_{[c]}$ （来自前一个 chunk）加上 chunk 内的累加：

$$S_{c,r} = S_{[c]} + \sum_{j=1}^r k_{c,j}^T v_{c,j}. \quad (2.18)$$

代回输出：

$$o_{c,r} = q_{c,r} S_{[c]} + q_{c,r} \sum_{j=1}^r k_{c,j}^T v_{c,j}. \quad (2.19)$$

第一项是 Q_c 与边界状态 $S_{[c]}$ 的乘积，对所有 r 并行；第二项是 chunk 内的因果 attention（因为求和上限是 r ，符合“只看过去”），可以写成 $((Q_c K_c^T) \odot \text{Lower}) V_c$ ，其中 Lower 是下三角（含对角线）掩码。

写成矩阵形式：

$$S_{[c+1]} = S_{[c]} + K_c^T V_c, \quad (2.20)$$

$$O_c = Q_c S_{[c]} + ((Q_c K_c^T) \odot \text{Lower}) V_c. \quad (2.21)$$

这就是 Linear Attention 的 chunk-wise 公式。两项分别对应“跨 chunk 状态贡献”和“chunk 内 attention”。没有 A 矩阵，也没有 U, W ，因为 Linear Attention 的更新是纯加法，没有 intra-chunk 的依赖需要消解。

注意 O_c 中的 $Q_c S_{[c]}$ 这一项， $S_{[c]}$ 在 c 之间是串行依赖的。所以 chunk-wise 并没有完全消除串行，而是把串行从 L 步降到 L/C 步。

2.15 GDN 的完整 chunk-wise 公式

GDN 比 Linear Attention 多了两件事：门控衰减 (α) 和 delta rule (β)。前者让旧状态在每步乘以 α_t 衰减，后者让写入变成“先纠错再相加”。我们设：

1. $\Gamma_c = \text{diag}(\gamma_{c,1}, \gamma_{c,2}, \dots, \gamma_{c,C})$ ，其中 $\gamma_{c,r} = \exp(g_{c,r}^{\text{cumsum}})$ 是 chunk 内累积门。
2. $B_c = \text{diag}(\beta_{c,1}, \beta_{c,2}, \dots, \beta_{c,C})$ ，是 delta rule 的写入强度。
3. γ_ℓ 是 chunk 边界处的累积门，即从序列开头到 chunk c 结束的总衰减。

GDN 的 chunk-wise 公式（来自 lab 页面）为：

$$A_c = \left(I + \text{StrictLower}(B_c \Gamma_c K_c K_c^T \Gamma_c^{\{-1\}}) \right)^{\{-1\}}, \quad (2.22)$$

$$U_c = A_c B_c V_c, \quad (2.23)$$

$$W_c = A_c B_c \Gamma_c K_c, \quad (2.24)$$

$$S_{[c+1]} = \gamma_\ell S_{[c]} + \gamma_\ell K_c^T \Gamma_c^{\{-1\}} (U_c - W_c S_{[c]}), \quad (2.25)$$

$$O_c = (1/\sqrt{d_k}) [\Gamma_c Q_c S_{[c]} + \Gamma_c \text{Lower}(Q_c K_c^T) \Gamma_c^{\{-1\}} (U_c - W_c S_{[c]})]. \quad (2.26)$$

公式里突然冒出来的 A, U, W 让人害怕，但它们的来历其实非常自然。下一节我们一步步把它们推出来。

2.16 逐步推导直觉：A、U、W 从何而来

2.16.1 A 矩阵的来源：因果依赖的逆

让我们先把 $\Gamma_c = B_c = I$ （无门、无 delta 强度），看 GDN 退化成 DeltaNet 的情形。递推为 $S_t = S_{t-1} + k_t^T (v_t - k_t S_{t-1})$ ，展开：

$$S_t = (I - k_t^T k_t) S_{t-1} + k_t^T v_t. \quad (2.27)$$

对 chunk 内 C 个 token，把每步的状态 $S_{c,r}$ 写成 $S_{[c]}$ （边界状态）和 chunk 内增量 $\Delta S_{c,r}$ 的组合。每一步 $\Delta S_{c,r}$ 依赖于前面所有 $\Delta S_{c,j}$ ($j < r$)，因为 $S_{c,r}$ 依赖于 $S_{c,r-1}$ ，而后者又依赖于 $S_{c,r-2}$ ，依此类推。

这种“严格的下三角依赖”恰好对应一个线性系统。设 chunk 内的“未纠错写入” $\tilde{U} = K_c^T V_c$ （也就是 Linear Attention 的版本），而真正经过 delta 纠错后的内部累积量 U_c 满足：

$$U_c + \text{StrictLower}(K_c K_c^T) U_c = \tilde{U}. \quad (2.28)$$

把 U_c 解出来：

$$U_c = (I + \text{StrictLower}(K_c K_c^T))^{\{-1\}} \tilde{U} = (I + \text{StrictLower}(K_c K_c^T))^{\{-1\}} K_c^T V_c. \quad (2.29)$$

对比 GDN 公式：当 $B_c = \Gamma_c = I$ 时， $A_c = (I + \text{StrictLower}(K_c K_c^T))^{\{-1\}}$ ， $U_c = A_c V_c = A_c B_c V_c$ （因为 $B_c = I$ ）。这正是 Linear Attention 公式里的 V_c ，被 delta 纠错机制“修正”后的版本。

直觉 A 矩阵就是“因果纠错的逆”。Linear Attention 把所有 $k_j^T v_j$ 直接加起来，DeltaNet 则要求“先把当前 token 对前面 token 的预测扣除，再加新 value”。这种“扣除”是因果的（只能扣前面的），所以对应一个严格下三角线性系统， A 就是这个系统的逆算子。

加上门和写入强度后， A 中间的 $K_c K_c^T$ 变成 $B_c \Gamma_c K_c K_c^T \Gamma_c^{\{-1\}}$ ： Γ 处理跨 token 的累积衰减， B 处理写入强度。但“严格下三角求逆”的结构不变。

2.16.2 U 与 W 的物理意义：预处理中间量

$U_c = A_c B_c V_c$ 的物理意义是：把 chunk 内的 V_c 经过 delta 纠错后的“纯内部累积”。它对应的是 Linear Attention 公式里的 $K_c^T V_c$ ，但每个 token 写入前都先扣除它对前面 token 的预测。

$W_c = A_c B_c \Gamma_c K_c$ 的物理意义是：chunk 内每个 token 的“有效 key”。它告诉我们“经过 delta 纠错后，当前 chunk 在 $S_{[c]}$ 上的真实写入模式是什么”。

更直观地说， $W_c S_{[c]}$ 这一项就是“如果 chunk 内不写入任何新 value，仅由 $S_{[c]}$ 经过 chunk 内的 delta 纠错后会变成什么”。所以 $U_c - W_c S_{[c]}$ 就是“chunk 内净增加的状态贡献”，再用 K_c^T 把它折回 $d_k \times d_v$ 状态空间。

记忆口诀： U 是“chunk 内 V 的纠错版”， W 是“chunk 内 K 的纠错版”。 A 是两者的公共“纠错算子”。一旦算出 A ， U 和 W 就只是它和 V, K 的乘积，可以并行算完。

2.16.3 状态递推如何结合 chunk 内信息

看 $S_{[c+1]}$ 的公式：

$$S_{[c+1]} = \gamma_\ell S_{[c]} + \gamma_\ell K_c^T \Gamma_c^{\{-1\}} (U_c - W_c S_{[c]}). \quad (2.30)$$

把它重新整理：

$$S_{[c+1]} = \gamma_\ell (I - K_c^T \Gamma_c^{\{-1\}} W_c) S_{[c]} + \gamma_\ell K_c^T \Gamma_c^{\{-1\}} U_c. \quad (2.31)$$

第一项 $\gamma_\ell (I - K_c^T \Gamma_c^{\{-1\}} W_c) S_{[c]}$ 是“边界状态经过 chunk 内 delta 纠错后的衰减版”。第二项 $\gamma_\ell K_c^T \Gamma_c^{\{-1\}} U_c$ 是“chunk 内新写入的累积”。两者都乘以 γ_ℓ ，因为 chunk 边界状态要在 chunk 之间继续衰减。

输出 O_c 的结构对称：

$$O_c = (1/\sqrt{d_k}) \left[\underbrace{\Gamma_c Q_c S_{[c]}}_{\text{跨 chunk 状态贡献}} + \underbrace{\Gamma_c \text{Lower}(Q_c K_c^T) \Gamma_c^{\{-1\}} (U_c - W_c S_{[c]})}_{\text{chunk 内 attention}} \right]. \quad (2.32)$$

第一项 $\Gamma_c Q_c S_{[c]}$ 对应 Linear Attention 里的 $Q_c S_{[c]}$ ，但乘上 Γ_c （每个 token 的衰减）。第二项对应 $((Q_c K_c^T) \odot \text{Lower}) V_c$ ，但 V_c 替换成了“纠错后的内部累积” $U_c - W_c S_{[c]}$ ，并补上 Γ 因子。

直觉 整个公式可以浓缩成一句话：“跨 chunk 部分用 $S_{[c]}$ 串行传递，chunk 内部分用 U_c 、 W_c 并行算出”。 A 一次性把 chunk 内所有 delta 纠错关系打包，剩下都是普通矩阵乘。

2.17 chunk size C 的选择权衡

chunk size C 是 chunk-wise parallel 唯一的超参，它同时决定并行度、串行步数和片上资源占用。我们看三种极端：

C 取值	串行步数	并行度	片上资源
$C = 1$	L	极低，退化为逐 token 递推	$d_k \times d_v$ 状态
$C = 64$ （实验值）	$L/64$	一次 64×64 矩阵乘，Tensor Core 友好	64×64 中间矩阵 + 状态
$C = L$	1	完全并行，但退化为 $L \times L$	$L \times L$ 中间矩阵爆炸

Table 12: chunk size C 的权衡

$C = 1$ 时 chunk-wise 公式退化为逐 token 递推，无法并行； $C = L$ 时退化为完全并行， $L \times L$ 矩阵爆炸。 $C = 64$ 是个甜点：

1. **并行度**：chunk 内的 $K_c K_c^T$ 、 A_c 、 $Q_c K_c^T$ 都是 64×64 的小矩阵，可以一次性塞进 SMEM，让 Tensor Core 高效运转。
2. **串行开销**： $L = 32768$ 时只需 512 步串行，相比 32768 步递推减少 64 倍。
3. **片上资源**： 64×64 的中间矩阵在 BF16 下是 8 KB，加上 A, U, W 等中间量总计仍在 SMEM 容量内。
4. **门控粒度**：累积门 $\gamma_{c,r}$ 在 chunk 内有 64 个值，对应一种“细到 token 级、粗到 chunk 级”的折中粒度。

实验中 $C = 64$ 是固定的，但理解它的来历很重要。如果你要适配不同的 d_k, d_v 或不同的 GPU（比如 SMEM 更小的卡）， C 是你首先要重新调的超参。

2.18 手算例子：用小数值走一遍 chunk-wise

为了让公式落地，我们用一组小数值手算 chunk 0 的 chunk-wise 流程。设 $d_k = d_v = 2$ ， $C = 2$ ， $\beta = 1$ ， $\alpha = 1$ （即 $B = \Gamma = I$ ， $\gamma_\ell = 1$ ，GDN 退化为 DeltaNet），初始状态 $S_{[0]} = 0$ 。

例 chunk 0 内有两个 token，输入为：

1. $k_1 = (1, 0)$ ， $v_1 = (3, 0)$ ， $q_1 = (1, 0)$

2. $k_2 = (1, 0)$, $v_2 = (10, 0)$, $q_2 = (1, 0)$
 写成矩阵: $K = \begin{pmatrix} 1 & 0 \\ 1 & 0 \end{pmatrix}$, $V = \begin{pmatrix} 3 & 0 \\ 10 & 0 \end{pmatrix}$, $Q = \begin{pmatrix} 1 & 0 \\ 1 & 0 \end{pmatrix}$ 。

第一步: 算 KK^T 与 A

$$KK^T = \begin{pmatrix} 1 & 0 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}. \quad (2.33)$$

严格下三角 $\text{StrictLower}(KK^T) = \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}$ (保留严格下三角, 对角线与上三角置零)。
 于是:

$$I + \text{StrictLower}(KK^T) = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}, \quad A = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}^{\{-1\}} = \begin{pmatrix} 1 & 0 \\ -1 & 1 \end{pmatrix}. \quad (2.34)$$

验证: $\begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ -1 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I$, 正确。

第二步: 算 U 与 W

$$U = AV = \begin{pmatrix} 1 & 0 \\ -1 & 1 \end{pmatrix} \begin{pmatrix} 3 & 0 \\ 10 & 0 \end{pmatrix} = \begin{pmatrix} 3 & 0 \\ 7 & 0 \end{pmatrix}. \quad (2.35)$$

$$W = AK = \begin{pmatrix} 1 & 0 \\ -1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}. \quad (2.36)$$

第三步: 算 $S_{[1]}$

$$S_{[1]} = K^T(U - WS_{[0]}) = K^T U = \begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} 3 & 0 \\ 7 & 0 \end{pmatrix} = \begin{pmatrix} 10 & 0 \\ 0 & 0 \end{pmatrix}. \quad (2.37)$$

第四步: 算 $O_{[0]}$

$$QK^T = \begin{pmatrix} 1 & 0 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}. \quad (2.38)$$

$\text{Lower}(QK^T) = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}$ (保留下三角含对角线)。

$$O_{[0]} = (1/\sqrt{2}) \text{Lower}(QK^T)(U - WS_{[0]}) = (1/\sqrt{2}) \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 3 & 0 \\ 7 & 0 \end{pmatrix} = (1/\sqrt{2}) \begin{pmatrix} 3 & 0 \\ 10 & 0 \end{pmatrix} \quad (2.39)$$

即 $o_1 = (3, 0)/\sqrt{2}$, $o_2 = (10, 0)/\sqrt{2}$ 。

用递推验证:

1. $S_1 = S_0 + k_1^T(v_1 - k_1 S_0) = \begin{pmatrix} 1 \\ 0 \end{pmatrix} (3, 0) = \begin{pmatrix} 3 & 0 \\ 0 & 0 \end{pmatrix}$
2. $S_2 = S_1 + k_2^T(v_2 - k_2 S_1) = \begin{pmatrix} 3 & 0 \\ 0 & 0 \end{pmatrix} + \begin{pmatrix} 1 \\ 0 \end{pmatrix} ((10, 0) - (1, 0) \begin{pmatrix} 3 & 0 \\ 0 & 0 \end{pmatrix}) = \begin{pmatrix} 3 & 0 \\ 0 & 0 \end{pmatrix} + \begin{pmatrix} 1 \\ 0 \end{pmatrix} (7, 0) = \begin{pmatrix} 10 & 0 \\ 0 & 0 \end{pmatrix}$
3. $o_1 = q_1 S_1 / \sqrt{d_k} = (1, 0) \begin{pmatrix} 3 & 0 \\ 0 & 0 \end{pmatrix} / \sqrt{2} = (3, 0) / \sqrt{2}$
4. $o_2 = q_2 S_2 / \sqrt{d_k} = (1, 0) \begin{pmatrix} 10 & 0 \\ 0 & 0 \end{pmatrix} / \sqrt{2} = (10, 0) / \sqrt{2}$

两种方法结果完全一致。注意 k_2 与 k_1 同方向, $v_2 = 10$ 覆盖了 $v_1 = 3$ (DeltaNet 的纠错特性), 而不是叠加成 13 (Linear Attention 的行为)。这正是 A 矩阵在做的事。

这个例子刻意选了 $k_1 = k_2$ 同方向, 让 KK^T 的非对角元素非零, A 才会偏离单位矩阵。如果 k_1, k_2 正交, $\text{StrictLower}(KK^T) = 0$, $A = I$, *chunk-wise* 退化成 *Linear Attention* 的简单形式, *delta* 纠错不发生作用。

2.19 GPU 存储层次与 Bank Conflict

2.20 引言：为什么存储层次决定 GDN 性能

上一章我们写出了第一个 TileLang baseline：四个 kernel 各做一件事，朴素直白。但如果你 profile 一下，会发现它在 GPU 上的效率远低于预期。原因不在算力，而在访存：GDN kernel 反复读写 K、A、S 这几个张量，如果每次都从 global memory 取，就要付出几百 cycle 的延迟代价。

直觉 把 GPU 想象成一家大工厂：global memory 是远郊的仓库，shared memory 是车间里的工具架，register 是工人手里的工具。每次都跑到远郊取料，工人大部分时间都在路上；把常用件搬到工具架上，干活才快。本章就是教你如何规划“工具架”。

本章的目标是把 GPU 的存储层次一次讲透，然后回到 GDN 这个具体场景，告诉你哪些张量该放哪里、shared memory 够不够用、bank conflict 怎么避免。后续章节的所有优化技巧，都建立在这一章的存储模型上。

2.21 GPU 存储层次总览

现代 GPU 有四级存储，从慢到快依次是：global memory (HBM)、L2 cache、shared memory、register。它们在容量、延迟、带宽、可见性上各有不同。

存储层级	容量	延迟	带宽	可见性
Global (HBM)	40-80 GB	400-600 cycle	1-2 TB/s	所有线程
L2 Cache	40-75 MB	200 cycle	3-5 TB/s	所有 SM 共享
Shared Memory	96-164 KB/SM	20-30 cycle	20-30 TB/s	同 SM 线程
Register	64 K/SM	1 cycle	最快	同线程

Table 13: GPU 存储层次对比（典型值，V100/A100）

可以看到，从 global 到 register，延迟相差几百倍，带宽也相差一个数量级以上。但容量也相应减小：register 比 global memory 小了六个数量级。所以优化的核心就是：把高频访问的数据放在更快的存储里，但又不能超出容量。

表格里的数字是典型值，具体型号会有差异。V100 的 HBM 是 32 GB，A100 是 40 或 80 GB；A100 的 shared memory 可以配到 164 KB/SM。写 kernel 时要以目标卡的 datasheet 为准。

2.22 Global Memory：高延迟的“远郊仓库”

Global Memory（全局内存）就是 GPU 板载的 **HBM**（High Bandwidth Memory，高带宽显存），所有线程都能访问。它的特点是容量大（40-80 GB）但延迟高（几百 cycle），峰值带宽在 1-2 TB/s 量级。

global memory 的性能高度依赖访问模式。最理想的访问是 **coalesced access**（合并访问）：同一个 warp（32 个线程）在同一时刻访问连续的地址，硬件可以把这 32 次访问合并成一次大事务。如果访问不连续（strided），实际带宽会大幅下降。

直觉 把 HBM 想象成一条很宽的高速公路（128 字节的车道），32 辆卡车并排走时只要他们正好对齐车道，就能一次运完。如果 32 辆车错位行驶，每辆车都要单独发一趟，高速公路就堵了。

在 GDN baseline 里，K、V、Q、A、S 都从 global memory 加载，如果每次 kernel 启动都重新 load 一遍，就会反复触发高延迟访问。后续优化的第一步就是把高频数据搬到 shared memory。

2.23 L2 Cache: 硬件管理的“二级缓存”

L2 Cache 是所有 SM 共享的硬件缓存，容量在 40-75 MB 量级（A100 是 75 MB）。它由硬件管理，程序员无法显式控制哪些数据驻留，但可以利用空间局部性让硬件自动缓存。

对 GDN 来说，如果一个 chunk 的 K、V 在多个阶段被反复读取，第二次访问很可能命中 L2，延迟从 400 cycle 降到 200 cycle。但 L2 的容量仍然有限，当 chunk 数很多、head 数很多时，L2 会被反复 evict。

不要把 L2 当成可靠的优化手段。它的命中率受整体访存模式影响，难以预测。在 TileLang 里我们能直接控制的是 shared memory，所以本章后续重点放在 shared memory 上。

2.24 Shared Memory: 程序员显式管理的高速缓存

Shared Memory（共享内存）是每个 SM 独有的一片高速 SRAM，容量在 96-164 KB/SM 量级，延迟 20-30 cycle，带宽可达 20-30 TB/s。与 L2 不同，shared memory 由程序员显式分配和管理，数据何时进、何时出完全可控。

直觉 Shared memory 就像车间里的工具架：每个车间（SM）独有，容量不大但拿取很快。工人（线程）之间通过工具架共享数据，不需要跑到仓库（global memory）。

2.24.1 大小配置

不同 GPU 的 shared memory 配置不同：

1. **V100**: 每 SM 96 KB，可在 64 KB shared + 32 KB L1 与 32 KB shared + 64 KB L1 之间配置。
2. **A100**: 每 SM 164 KB shared + 32 KB L1，配置更慷慨。

写 kernel 时需要确认目标卡的 shared memory 上限。如果某个 block 申请的 shared memory 超过上限，kernel 启动会失败。另外，每个 block 默认可用的 shared memory 上限不一定等于 SM 总量，V100 默认每 block 最多 48 KB，需要用 `cudaFuncSetAttribute` 显式提升到 96 KB。

2.24.2 在 TileLang 中分配

TileLang 提供 `T.alloc_shared` 来分配 shared memory。基本用法：

```
K_shared = T.alloc_shared([C, dk], "bfloat16")
V_shared = T.alloc_shared([C, dv], "bfloat16")
T.copy(K[b, n, :, h, :], K_shared)
```

数据加载到 shared memory 后，后续所有访问都是低延迟的。注意 `T.copy` 仍然是从 global 到 shared 的一次完整搬运，所以要确保 shared 里的数据被复用足够多次，才能摊薄这次搬运的开销。在 GDN 里，`K` 在阶段 2 和阶段 3 都被用到，加载一次复用两次，已经很划算；如果再加上阶段间的融合，复用次数更多。

2.25 Bank Conflict: shared memory 的隐形杀手

Shared memory 虽然快，但有一个隐形杀手：**bank conflict**（bank 冲突）。Shared memory 被划分为 32 个 bank，每 bank 宽 4 字节（32 bit）。一个 warp 有 32 个线程，理想情况下每个线程访问不同的 bank，32 个访问可以一次完成。

如果两个或更多线程同时访问同一个 bank 的不同地址，就会发生 bank conflict，硬件只能把这些访问串行化，延迟成倍增加。最严重的情况是 32-way bank conflict，shared memory 的有效带宽降到 1/32。

直觉 把 32 个 bank 想象成 32 个收银台。每个线程排到不同的收银台，所有人都能同时结账；如果 32 个人全挤到一个收银台，就要排 32 次队。

2.25.1 32-bit stride 访问的常见陷阱

一个常见陷阱是 stride 访问。假设你有一个 $[C, d]$ 的矩阵，每个线程访问 `mat[thread_id, 0]`，那么相邻线程访问的地址间隔是 $d \times 2$ 字节（BF16 情况下）。如果 d 是 32 的倍数（且元素按 32-bit word 对齐），所有线程会落到同一个 bank。

具体规则：对 32-bit 访问，只要 stride 不是 32 的倍数就不会有冲突。BF16 是 16-bit，两个相邻 BF16 组成一个 32-bit word，规律略有不同但思路一致。最简单的判断方式是看“相邻线程访问地址的最低 5 位（bank id）是否相同”。

2.25.2 解决方法：padding 与交错访问

常见的解决方法有两种：

1. **Padding**：在矩阵的列维度上加一列 padding，让 stride 不是 bank 数的倍数。例如把 $[64, 128]$ 改成 $[64, 129]$ ，多出来的一列不存数据，但能避免 bank conflict。
2. **交错访问**：调整线程到数据的映射，让相邻线程访问不同的 bank。TileLang 里通过 swizzle layout 实现，本质是把 thread 到 element 的映射改写一下。

TileLang 的 `T.alloc_shared` 不自动 padding，需要你在写 kernel 时手动考虑访问模式。如果 profile 发现 shared memory 利用率很低，第一个怀疑对象就是 bank conflict。

2.26 Register: 每线程私有的最快存储

Register（寄存器）是每线程私有的存储，延迟 1 cycle，是 GPU 上最快的存储。但寄存器数量有限：V100 每 SM 64K 个寄存器，每线程最多 255 个。

如果 kernel 用的寄存器太多，会导致 **register pressure**（寄存器压力），进而降低 **occupancy**（占用率），即每个 SM 上能同时跑的 warp 数减少。Occupancy 下降意味着延迟隐藏能力变差，性能反而退化。

直觉 Register 就像工人手里的工具，越多越方便，但手就那么大，拿太多工具反而手忙脚乱。寄存器分配要在“够用”和“不浪费”之间找平衡。

在 GDN 里，中间标量（如 γ 、 β ）通常放 register；矩阵片段（tile）的累加器也放 register。TileLang 用 `T.alloc_fragment` 分配 register 级别的存储：

```
acc = T.alloc_fragment([C, dv], "float32")
T.clear(acc)
T.gemm(A_shared, V_shared, acc)
```

注意累加器一定要用 FP32，否则 BF16 累加多次会累积不可接受的误差。这也是为什么 output 容差 $1e-2$ 看起来宽：BF16 的有效位决定了它无法更紧。

2.27 在 GDN 中的应用：各张量该放哪里

有了存储层次的全景图，我们可以回到 GDN，规划每个张量应该放在哪一级存储。原则是：高频访问的数据放 shared memory，单线程私有的标量放 register，低频大张量留在 global memory。

张量	大小（每 chunk）	建议位置	原因
K	$C \times d_k = 64 \times 128$	Shared	阶段 2、3 复用
V	$C \times d_v = 64 \times 128$	Shared	阶段 1、3 复用
Q	$C \times d_k = 64 \times 128$	Shared	阶段 4 用
A	$C \times C = 64 \times 64$	Shared	阶段 1、2 复用
gamma, beta	C	Register	标量，按行作用
S	$d_k \times d_v = 128 \times 128$	Shared 分块或 global	FP32 时 64 KB，需分块
U, W	中间结果	Register 或 Shared	避免写回 global

Table 14: GDN 各张量的存储位置规划

几个关键决策点：

1. K、V、Q：每个 chunk 的 $K/V/Q$ 加载一次到 shared，多个阶段复用，避免重复 global 访问。
2. A 矩阵： 64×64 在 BF16 下只有 8 KB，常驻 shared 完全负担得起。
3. S 矩阵： 128×128 在 FP32 下是 64 KB，单独一个 block 的 shared 装不下，需要分块（tiling）或 ping-pong 缓冲。
4. 中间量 U、W：在 shared 或 register 里算完直接给下一阶段用，不写回 global，能省一次往返。

S 的处理是最棘手的。一种常见策略是把它分成 32×32 的小块，每次只在 shared memory 里保留一块或两块（ping-pong），算完就替换。这样 shared 占用降到 $2 \times 32 \times 32 \times 4 = 8\text{KB}$ ，可控得多。

2.28 Kernel Fusion 与 Ping-pong 原理

2.29 引言：朴素实现的困境

GDN (Gated Delta Network, 门控 Delta 网络) 的前向计算可以被拆成若干阶段：先算 $U = ABV$ ，再算 $W = AB\Gamma K$ ，然后做状态 S 的 chunk 递推，最后计算输出 O 。如果你用最朴素的方式实现，每个阶段写成一个独立的 **kernel** (CUDA 核函数)，那么中间结果 U 、 W 、 S 都要先写回 **global memory** (全局显存)，下一个 kernel 再从 global memory 读回来。

这样做有两个问题。第一，中间结果在 global memory 上往返，产生了大量不必要的访存流量，而 GPU 的显存带宽是稀缺资源。第二，每启动一个 kernel 都有固定的 **kernel launch overhead** (核函数启动开销)，kernel 数量一多，开销就累积起来，计算单元在等待中闲置。本章我们就来解决这两个问题：用 **kernel fusion** (核函数融合) 减少访存，用 **ping-pong buffer** (乒乓缓冲) 重叠计算与访存。

2.30 Kernel Launch 开销

我们先看第一个问题。在 CUDA 中，每次启动一个 kernel，主机端需要通过驱动把参数和指令送到 GPU，GPU 端也要做调度和初始化。这个过程有一个固定的延迟，通常在几个微秒的量级。

直觉 你可以把 kernel launch 想象成打电话：每次拨号都要等几秒接通，不管你只说一句话还是聊一小时，接通的等待时间是一样的。如果你的每句话都挂掉重打，光是拨号就浪费大量时间。

对于 GDN 的前向，如果 U 、 W 、 S 、 O 各一个 kernel，再加上各种辅助 kernel，可能要启动十几个 kernel。假设每次 launch 需要 $5\mu s$ ，十几次就是 $50\mu s$ ，对于一个大模型里被反复调用的层来说，这是一笔不小的浪费。更严重的是，kernel 之间无法重叠：前一个 kernel 必须完成并把结果写回 global memory，后一个 kernel 才能开始读。

你可以用 NVIDIA Nsight Systems (nsys) 来 profiling，在 timeline 上会看到 kernel 之间的空隙 (gap)，那就是 launch 开销和 global memory 往返的等待。空隙越密集，说明你的 kernel 切分越碎，优化空间越大。

2.31 中间结果的访存代价

我们再算一笔账，看看中间结果在 global memory 上往返到底要搬多少字节。设批大小 B ，序列长度 T ，头数 H_v ，key 维度 d_k ，value 维度 d_v ，数据类型为 FP16 (每个元素 2 字节)。

中间量	形状	元素数	字节数
U	$B \times T \times H_v \times d_v$	$BTH_v d_v$	$2BTH_v d_v$
W	$B \times T \times H_v \times d_k$	$BTH_v d_k$	$2BTH_v d_k$
S	$B \times H_v \times d_k \times d_v$	$BH_v d_k d_v$	$2BH_v d_k d_v$

每个中间量都要被写一次、读一次，所以 global memory 的往返流量是字节数的两倍。

例 取一组小数值： $B = 1$ ， $T = 512$ ， $H_v = 4$ ， $d_k = d_v = 64$ ，FP16。

U 的字节数： $2 \times 1 \times 512 \times 4 \times 64 = 262144$ ，即 256 KB。

W 的字节数：同样 256 KB。

S 的字节数： $2 \times 1 \times 4 \times 64 \times 64 = 32768$ ，即 32 KB。

朴素实现下， U 要写一次再读一次，往返 512 KB； W 同样 512 KB； S 在递推中每个 chunk 都要读写，往返更多。仅 U 和 W 两项就超过 1 MB 的无谓流量，而这还只是一层、一个小批次。

注意， S 的递推是跨 chunk 的，每个 chunk 都要更新 S ，所以 S 的访存流量与 chunk 数量成正比，实际比 U 、 W 更频繁。

2.32 Kernel Fusion 原理

直觉 既然 U 算完马上就要被下一步用，何必先存到 global memory 再读回来？不如在同一个 kernel 里，算完 U 直接留在片上（shared memory 或寄存器），紧接着算下一步。这就是 kernel fusion 的核心思想。

Kernel fusion（核函数融合）指的是把多个原本独立的 kernel 合并成一个，让中间结果留在片上存储里，避免 global memory 的往返。关键在于：哪些阶段可以融合，哪些不行。

2.32.1 可以融合的部分

$U = ABV$ 和 $W = AB\Gamma K$ 共享输入 A 、 B ，而且都只依赖当前 chunk 的数据，没有跨 chunk 依赖。因此我们可以在同一个 kernel 里，先把 A 、 B 、 V 、 Γ 、 K 一次性 load 进 shared memory，然后同时计算 U 和 W ，它们都留在片上。

同样， S 的递推 $S_t = A_t S_{\{t-1\}} + W_t$ 和输出 O 的计算在每个 chunk 内部是连续的，算完 S_t 立刻可以参与 O 的计算，不需要落地到 global memory。

这其实就是 FlashAttention 系列的核心洞察：把 attention 拆成 chunk，chunk 内部融合，chunk 之间串行递推。FlashQLA、FLA、FlashInfer 等工作把这个思路推广到了线性注意力和各种门控递归结构。GDN 的优化与之同构。

2.32.2 不能融合的部分

跨 chunk 的 S 依赖是串行的：chunk c 的 S_c 依赖 chunk $c-1$ 的 $S_{\{c-1\}}$ 。你不可能在 $S_{\{c-1\}}$ 还没算完时就开始算 S_c 。所以不同 chunk 的 S 递推不能并行融合，只能顺序处理。

但这不妨碍我们在每个 chunk 内部做融合：chunk c 内部，算 U 、 W 、更新 S 、算 O 可以打包成一个 kernel，chunk 之间串行循环。

具体来说，融合后的 kernel 结构是一个外层循环遍历各 chunk，循环内部依次执行 load 数据、算 U 和 W 、递推 S 、算 O 。整个前向只需启动一个 kernel，launch 开销从十几次降为一次。

2.33 等价数学变换

有时候直接融合会遇到困难：某个中间量太大，片上存不下，或者计算顺序导致必须先算完一个完整的大矩阵。这时候我们可以做等价数学变换，改变计算顺序来减少中间量的大小。

直觉 回想结合律： $(AB)C = A(BC)$ 。如果你先算 AB ，得到的中间矩阵可能很大；但如果你先算 BC ，中间矩阵可能小得多。选择哪个结合方式，取决于哪个中间量更小、更容易留在片上。

在 GDN 中， $U = ABV$ 可以选择不同的乘法顺序。如果 A 和 B 先乘，中间结果是 $[B, T, H_v, H_v]$ ；如果 B 和 V 先乘，中间结果是 $[B, T, H_v, d_v]$ 。当 H_v 远大于 d_v 时，后者更省显存。这类变换的目的是让中间量尽可能小，从而能放进 shared memory 或寄存器。

2.34 ping-pong buffer：双缓冲原理

融合解决了 chunk 内部的访存问题，但 chunk 之间的串行递推仍然有瓶颈：每个 chunk 都要先从 global memory load 数据，再计算。如果 load 完才算，计算单元在 load 期间闲置；如果算完才 load 下一个 chunk 的数据，访存单元在计算期间闲置。

直觉 想象你在洗衣服：洗衣机洗一批（计算），同时你可以去收下一批脏衣服（load）。如果你只有一双手，洗完一批才能去拿下一批，中间手在闲。但如果你有两个篮子，一个在洗的时候，另一个已经装好下一批，洗完直接换篮子，中间不闲着。这就是 ping-pong buffer。

ping-pong buffer（乒乓缓冲，也叫 double buffer，双缓冲）用两块缓冲区交替工作：

1. 在 chunk c 的计算阶段，consumer 从 buffer0 读数据计算，同时 producer 把 chunk $c + 1$ 的数据 load 进 buffer1。
2. 计算完成后，两个 buffer 交换角色：下一轮从 buffer1 计算，buffer0 接收 chunk $c + 2$ 的数据。
3. 如此往复，计算和访存重叠。

这样，只要 load 的时间不超过计算的时间，访存延迟就被完全隐藏在计算之中，GPU 利用率大幅提升。

2.35 同步与正确性

双缓冲看起来美好，但有一个关键问题：producer 还没把 chunk $c + 1$ 的数据写完时，consumer 不能开始读 buffer1；consumer 还没读完 buffer0 时，producer 不能往 buffer0 写 chunk $c + 2$ 的数据，否则会覆盖正在用的内容。

这就需要同步机制。CUDA 提供了几种方式：

1. **barrier**（屏障）：producer 和 consumer 都到达 barrier 后才继续，保证双方都完成各自阶段。
2. **shared memory flag**（共享内存标志位）：producer 写完后置 flag，consumer 轮询 flag，看到后才读。
3. **cooperative group**（协作组）的同步原语：更灵活的 warp 级或 block 级同步。

同步不能太频繁，否则又变成串行；也不能太松，否则会读到脏数据。合理的做法是在每个 chunk 边界做一次同步，chunk 内部尽量让 producer 和 consumer 各自独立运转。

直觉 同步就像十字路口的红绿灯：太频繁（每个路口都停）则通行效率低，太少（没有红绿灯）则容易撞车。合理的做法是在主干道交汇处设一个红绿灯，中间路段自由通行。chunk 边界就是那个交汇路口。

2.36 multi-buffering: 更深的流水线

两个 buffer 是最简单的双缓冲，但有时候访存延迟比计算延迟长，两个 buffer 不够用。这时候可以推广到 **multi-buffering**（多缓冲），使用 3 个或更多 buffer，构成更深的流水线。

直觉 双缓冲像两条传送带交替，三缓冲像三条。如果 load 一次要 3 个计算周期那么久，你需要至少 3 个 buffer 在排队，才能保证每个计算周期都有数据可用，不让计算空等。

多缓冲的代价是占用了更多 shared memory 或 register，可能降低 **occupancy**（占用率，即同时活跃的 warp 数）。所以 buffer 数量不是越多越好，需要在隐藏延迟和保持 occupancy 之间权衡。

经验法则： $buffer$ 数量至少要等于 $\left\lceil \frac{t_{load}}{t_{compute}} \right\rceil$ ，其中 t_{load} 是一次 load 的延迟， $t_{compute}$ 是一次计算的时间。如果 load 要 $6\mu s$ ，计算要 $3\mu s$ ，那么至少需要 2 个 buffer；如果 load 要 $9\mu s$ ，计算要 $3\mu s$ ，则需要 3 个 buffer。

2.37 手算例子：时间线对比

例 设我们有 4 个 chunk，每个 chunk 的数据 load 需要 $2\mu s$ ，计算需要 $3\mu s$ 。

朴素串行：每个 chunk 先 load 再算，load 和计算不重叠。

时刻	0-2	2-5	5-7	7-10
动作	load chunk 0	算 chunk 0	load chunk 1	算 chunk 1

时刻	10-12	12-15	15-17	17-20
动作	load chunk 2	算 chunk 2	load chunk 3	算 chunk 3

总时间： $4 \times (2 + 3) = 20\mu s$ ，其中计算只占 $12\mu s$ ，有 $8\mu s$ 在等 load。

ping-pong 双缓冲：load 和计算重叠。

时刻	0-2	2-5	5-8	8-11
buffer0	load chunk 0	算 chunk 0	load chunk 2	算 chunk 2
buffer1	-	load chunk 1	算 chunk 1	load chunk 3

时刻	11-14	动作
算 chunk 3		

总时间：2（首个 load）+4×3（计算，与后续 load 重叠）=14 μ s。比朴素快了 6 μ s，访存延迟被大部分隐藏。

注意最后一步 chunk 3 只有计算没有后续 load，所以末尾有一个 load 空窗，这是流水线不可避免的填充和排空开销。

2.37.1 加速比分析

从上面的例子可以归纳出一般规律。设 load 延迟为 L ，计算延迟为 C ，chunk 数为 N 。朴素串行的总时间为 $N(L+C)$ 。ping-pong 在理想情况下，除了第一个 chunk 的 load 不可隐藏之外，后续每个 chunk 的 load 都与上一个 chunk 的计算重叠，总时间为 $L+NC$ （当 $L < C$ 时）。

加速比为 $\frac{N(L+C)}{L+NC}$ 。当 N 很大时，加速比趋近于 $\frac{L+C}{C} = 1 + \frac{L}{C}$ 。也就是说，load 延迟相对于计算越大，ping-pong 的收益越高。但如果 $L > C$ （load 比计算还慢），那么计算单元会等 load，ping-pong 也无法完全隐藏，这时候就需要更多的 buffer（multi-buffering）。

2.38 Warp Specialization 与 Tensor Core

2.39 引言：GDN 的两类计算

GDN 的前向计算里，既有矩阵乘法，又有逐元素运算。矩阵乘法如 QK^T 、 $U=ABV$ 、 qS ，属于密集的线性代数运算；逐元素运算如 $\exp(g_{\text{cumsum}})$ 、 Γ 的逐点乘法、 β 缩放，属于标量级别的操作。如果我们让 GPU 串行执行这些操作，Tensor Core 算矩阵乘时 CUDA Core 闲着，CUDA Core 算逐元素时 Tensor Core 闲着，两类计算单元交替空闲，效率低下。

本章我们讨论如何用 **warp specialization**（warp 专一化）把不同 warp 分配到不同角色，让 CUDA Core 和 Tensor Core 真正并行工作，把逐元素运算隐藏在 Tensor Core 的矩阵乘背后。

2.40 GPU 的两类计算单元

现代 GPU 上有两类计算单元，它们的用途和特性截然不同。

2.40.1 CUDA Core：标量逐元素计算

CUDA Core 是 GPU 上最基础的标量计算单元，负责逐元素的加减乘除、超越函数（如 $\exp$ 、 $\log$ 、 $\tanh$ ）以及逻辑运算。每个 CUDA Core 一次处理一个元素的标量操作，精度可以是 FP32、FP64 或 INT。

直觉 CUDA Core 像是一个全能但单件的工匠：什么活都能干，但一次只做一件。算 $\exp$ 、做逐点乘法，都是它的拿手好戏，只是速度不如专门做矩阵乘的 Tensor Core。

2.40.2 Tensor Core：矩阵乘加

Tensor Core 是专门为矩阵乘加设计的硬件单元。一条 **MMA**（Matrix Multiply-Accumulate，矩阵乘加）指令可以完成一个 $m \times n \times k$ 的小矩阵块乘加，例如 $16 \times 16 \times 16$ ，一次性完成 16×16 个输出元素的 16 次乘加。

以 V100 (sm_70) 为例，它的 Tensor Core 支持 FP16/BF16 精度的 $16 \times 16 \times 16$ MMA 操作。相比于用 CUDA Core 一个元素一个元素地算，Tensor Core 的吞吐量高出一个数量级以上。

Tensor Core 的工作方式是： $D = A \times B + C$ ，其中 A 、 B 、 C 、 D 都是片段 (fragment)，寄存器里的一小块矩阵。一次 MMA 把整个片段更新完毕。

2.40.3 两类单元的吞吐对比

我们可以粗略估算一下差距。一个 $16 \times 16 \times 16$ 的 FP16 矩阵乘加，需要 $16 \times 16 \times 16 = 4096$ 次乘加。如果用 CUDA Core，需要 4096 个周期才能完成（假设一个 CUDA Core 一个周期做一次乘加）。而 Tensor Core 只需一条 MMA 指令，在几个周期内完成这 4096 次乘加，加速比高达数百倍。这就是为什么我们要尽量让矩阵乘走 Tensor Core。

2.41 Tensor Core 编程

直接用 Tensor Core 需要通过 **wmma** (Warp Matrix Multiply-Accumulate) API，加载片段、调用 `mma_sync`、存储结果。这比较底层，写起来繁琐。在实验中，我们通常用更高层的抽象。

直觉 你可以把 `wmma` 想象成手动挡：你要自己挂挡、踩离合。而 `TileLang` 里的 `T.gemm` 像自动挡：你告诉它算哪两个矩阵的乘法，它帮你调度到 Tensor Core 上。

在 **TileLang**（一种 GPU kernel 编程 DSL）中，调用 `T.gemm(...)` 就会自动映射到 Tensor Core 的 MMA 指令。类似地，**CUTLASS** 和 **Triton** 也提供了高层抽象来使用 Tensor Core，让开发者不必手写 `wmma` 的片段管理。

`wmma` API 的基本流程是三步：用 `load_matrix_sync` 把数据从 *shared memory* 加载到寄存器片段，用 `mma_sync` 执行矩阵乘加，再用 `store_matrix_sync` 把结果写回 *shared memory*。高层 DSL 帮你把这三步封装起来，你只需声明要算哪个 `gemm`。

2.42 Warp: GPU 的执行单元

在讲 warp specialization 之前，我们需要理解 **warp**（线程束）是什么。GPU 上最小的执行单位不是一个线程，而是一个 warp，包含 32 个线程。同一个 warp 内的线程以 **SIMT** (Single Instruction Multiple Threads，单指令多线程) 方式锁步执行：它们在同一时刻执行同一条指令，只是操作各自的数据。

直觉 把 warp 想象成一队 32 个士兵，教官喊一个口令，所有人同时做同一个动作，只是每个人手里的材料不同。你没法让一个 warp 里的两个线程同时做不同的事，但你可以让不同 warp 做不同的事。

这个特性正是 warp specialization 的基础：既然不同 warp 可以做不同的事，我们就可以让一些 warp 专门 load 数据，另一些 warp 专门做计算。

2.43 Warp Specialization 思想

warp specialization (warp 专一化) 的核心思想是分工：把一个 block 里的不同 warp 分配到不同角色，让它们并行工作。

2.43.1 Producer 与 Consumer

通常我们把 warp 分成两类：

1. **producer warp** (生产者 warp)：负责从 global memory 把数据（如 K 、 V 、 A ）load 到 shared memory，产生数据供 consumer 使用。
2. **consumer warp** (消费者 warp)：负责从 shared memory 读数据，执行计算（如 gemm、逐元素运算），消费 producer 提供的数据。

producer 和 consumer 并行运转：当 consumer 在算 chunk c 时，producer 在 load chunk $c + 1$ 的数据。这与上一章的 ping-pong buffer 天然配合：producer 往一块 buffer 写，consumer 从另一块 buffer 读。

warp specialization 本质上是把 ping-pong 的“访存与计算重叠”从时间维度细化到了 warp 级别：不是同一个 warp 先 load 后算，而是不同 warp 各管一摊。

2.43.2 warp 数量的分配

一个 block 通常有 4 到 8 个 warp。如何分配取决于访存和计算的比例。如果访存是瓶颈，可以多分配几个 warp 做 producer；如果计算是瓶颈，就把大部分 warp 留给 consumer。一个常见的分配是：1 个 producer warp 负责搬运，3 到 7 个 consumer warp 负责计算。这样 producer 可以持续供给数据，consumer 有足够的并行度来跑 Tensor Core。

2.44 在 GDN 中的应用

在 GDN 的前向中，我们可以这样分工：

1. 矩阵乘 (QK^T 、 $U = ABV$ 、 qS)：交给 Tensor Core，由 consumer warp 执行 `T.gemm`。
2. 逐元素运算 (exp、 Γ 乘法、 β 缩放)：交给 CUDA Core，可以让一个 consumer warp 专门做这些标量操作。
3. 数据搬运 (K 、 V 、 A 、 B load)：交给 producer warp，与计算重叠。

关键在于，Tensor Core 的矩阵乘和 CUDA Core 的逐元素运算可以同时进行。当我们把逐元素运算交给一个 warp，把矩阵乘交给另一个 warp 跑 Tensor Core 时，CUDA Core 的活就被隐藏在 Tensor Core 的计算时间里了。这种分工让两类计算单元不再交替空闲，而是同时满载工作。

直觉 想象一个厨房：主厨用烤箱烤肉 (Tensor Core 算矩阵乘，耗时但产出大)，助手在旁边切菜 (CUDA Core 算逐元素，快但琐碎)。如果主厨一边烤肉一边自己切菜，就得停下手里的活。但如果有助手专门切菜，主厨只管烤肉，切菜的活就被隐藏在烤肉的时间里了。

2.44.1 为什么要分两个 consumer warp

你可能会问：既然都是计算，为什么不把 Tensor Core 和 CUDA Core 的活交给同一个 warp？原因是同一个 warp 在同一时刻只能执行一条指令，要么跑 MMA（走 Tensor Core），要么跑逐元素（走 CUDA Core），两者无法并行。只有把它们分到不同 warp，让 GPU 的调度器把它们分派到不同的执行端口，才能真正同时执行。这就是 warp specialization 能带来重叠收益的根本原因。

2.45 同步机制

producer 和 consumer 共享 shared memory，所以它们之间必须有同步机制。producer 写完一块 buffer 后要通知 consumer 可以读了，consumer 读完一块 buffer 后要通知 producer 可以覆盖了。

常用的同步方式：

1. **barrier**（屏障）：producer 和 consumer 都到达 barrier 后继续，简单但粒度粗。
2. **shared memory flag**（标志位）：producer 写完后置 flag，consumer 轮询，更灵活但需要小心内存可见性。
3. **named barrier**（命名屏障）：CUDA 允许只同步指定的 warp 子集，减少不必要的等待。

与 double buffer 配合时，典型流程是：producer 往 buffer1 写 chunk $c + 1$ 的数据，consumer 从 buffer0 读 chunk c 的数据。两者完成后交换，进入下一轮。同步点设在每轮交换处，chunk 内部各自独立运转。

在 CUDA 中，`__syncthreads()` 会同步 block 内所有线程，粒度较粗。如果只想同步 producer 和 consumer 两组 warp，可以用 `cuda::barrier`（CUDA 11 引入的同步原语）或 `named barrier`，减少不必要的等待。TileLang 等高层 DSL 通常会自动处理这些同步细节。

2.46 手算例子：重叠收益

例 设每个 chunk 的计算包含两部分：Tensor Core 的矩阵乘需要 $10\mu s$ ，CUDA Core 的逐元素运算需要 $5\mu s$ 。我们处理 3 个 chunk。

朴素串行：每个 chunk 先算 Tensor Core 再算 CUDA Core，两者不重叠。

chunk	Tensor Core	CUDA Core	小计
0	$10\mu s$	$5\mu s$	$15\mu s$
1	$10\mu s$	$5\mu s$	$15\mu s$
2	$10\mu s$	$5\mu s$	$15\mu s$

总时间： $3 \times 15 = 45\mu s$ 。

Warp specialization 重叠：Tensor Core 和 CUDA Core 并行，CUDA Core 的 $5\mu s$ 被隐藏在 Tensor Core 的 $10\mu s$ 中。

chunk	重叠后耗时	说明
-------	-------	----

0	10 μ s	CUDA Core 与 Tensor Core 并行
1	10 μ s	同上
2	10 μ s	同上

总时间： $3 \times 10 = 30\mu\text{s}$ ，节省了 $15\mu\text{s}$ ，相当于 33% 的加速。CUDA Core 的工作被完全隐藏，瓶颈变成了 Tensor Core。

2.46.1 多 chunk 的流水线效果

上例只考虑了单种计算的 chunk。实际中，每个 chunk 还包含数据 load（由 producer warp 负责）。结合上一章的 ping-pong buffer，producer warp 在 consumer 算 chunk c 时 load chunk $c + 1$ ，访存延迟也被隐藏。这样三层重叠（Tensor Core 计算、CUDA Core 逐元素、producer 数据搬运）同时进行，GPU 的利用率被推到最高。

如果 CUDA Core 的工作比 Tensor Core 还长（比如 $12\mu\text{s}$ 对 $10\mu\text{s}$ ），那么 CUDA Core 成为瓶颈，Tensor Core 反而被隐藏。warp specialization 的收益取决于能否把较短的那个隐藏掉。

2.47 寄存器压力与 occupancy

warp specialization 不是没有代价的。当不同 warp 承担不同角色时，每个 warp 可能需要各自的一套寄存器来保存状态，block 总的寄存器使用量上升。

直觉 GPU 上每个 block 能用的寄存器总量是固定的（比如 SM 上 64K 个寄存器）。如果每个 block 用太多寄存器，SM 上能同时驻留的 block 数就少了，也就是 **occupancy**（占用率）下降。occupancy 低意味着 warp 调度器没有足够的 warp 来隐藏延迟，性能反而可能变差。

所以在设计 warp specialization 时，要平衡分工带来的并行收益和寄存器压力带来的 occupancy 损失。常见的做法是限制 producer warp 的数量（比如只用 1 到 2 个 warp 做 load），把更多寄存器留给 consumer warp 做计算。

直觉 这就像公司人员配置：如果搬运工（producer）太多，办公室（寄存器）就挤不下足够的工程师（consumer），反而效率下降。关键是找到搬运工恰好够用、工程师尽量多的平衡点。

2.47.1 如何判断 occupancy 是否足够

判断 occupancy 是否成为瓶颈，可以用 NVIDIA Nsight Compute (ncu) 查看 **achieved occupancy**（实际占用率）。如果实际占用率远低于理论最大值，说明寄存器或 shared memory 限制了活跃 warp 数。这时候可以考虑减少每个 warp 的寄存器使用（比如把一些中间量存到 shared memory 而非寄存器），或者减少 producer warp 的数量。反之，如果实际占用率接近 100%，说明 occupancy 不是瓶颈，可以放心地增加分工。

